OGC GeoSPARQL - A Geographic Query Language for RDF Data

```
<style>
  .imageblock > .title {
    text-align: inherit;
  }
</style>
```

Open Geospatial Consortium

Submission Date: <yyyy-mm-dd>

Approval Date: <yyyy-mm-dd>

Publication Date: <yyyy-mm-dd>

External identifier of this OGC® document: http://www.opengis.net/doc/IS/geosparql/1.1

Internal reference number of this OGC® document: YY-DOC

Version: 1.1

Category: OGC® Implementation Specification

Editors: Matthew Perry, John Herring, Nicholas J. Car, Timo Homburg & Simon J.D. Cox

OGC GeoSPARQL - A Geographic Query Language for RDF Data

Copyright notice

Copyright © 2021 Open Geospatial Consortium

To obtain additional rights of use, visit http://www.opengeospatial.org/legal/

Warning

This document is an OGC Member approved international standard. This document is available on a royalty free, non-discriminatory basis. Recipients of this document are invited to submit, with their comments, notification of any relevant patent rights of which they are aware and to provide supporting documentation.

Document type: OGC® Standard

Document subtype: Encoding

Document stage: Approved for Public Release

Document language: English

License Agreement

Permission is hereby granted by the Open Geospatial Consortium, ("Licensor"), free of charge and subject to the terms set forth below, to any person obtaining a copy of this Intellectual Property and any associated documentation, to deal in the Intellectual Property without restriction (except as set forth below), including without limitation the rights to implement, use, copy, modify, merge, publish, distribute, and/or sublicense copies of the Intellectual Property, and to permit persons to whom the Intellectual Property is furnished to do so, provided that all copyright notices on the intellectual property are retained intact and that each person to whom the Intellectual Property is furnished agrees to the terms of this Agreement.

If you modify the Intellectual Property, all copies of the modified Intellectual Property must include, in addition to the above copyright notice, a notice that the Intellectual Property includes modifications that have not been approved or adopted by LICENSOR.

THIS LICENSE IS A COPYRIGHT LICENSE ONLY, AND DOES NOT CONVEY ANY RIGHTS UNDER ANY PATENTS THAT MAY BE IN FORCE ANYWHERE IN THE WORLD.

THE INTELLECTUAL PROPERTY IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NONINFRINGEMENT OF THIRD PARTY RIGHTS. THE COPYRIGHT HOLDER OR HOLDERS INCLUDED IN THIS NOTICE DO NOT WARRANT THAT THE FUNCTIONS CONTAINED IN THE INTELLECTUAL PROPERTY WILL MEET YOUR REQUIREMENTS OR THAT THE OPERATION OF THE INTELLECTUAL PROPERTY WILL BE UNINTERRUPTED OR ERROR FREE. ANY USE OF THE INTELLECTUAL PROPERTY SHALL BE MADE ENTIRELY AT THE USER'S OWN RISK. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR ANY CONTRIBUTOR OF INTELLECTUAL PROPERTY RIGHTS TO THE INTELLECTUAL PROPERTY BE LIABLE FOR ANY CLAIM, OR ANY DIRECT, SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES, OR ANY DAMAGES WHATSOEVER RESULTING FROM ANY ALLEGED INFRINGEMENT OR ANY LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR UNDER ANY OTHER LEGAL THEORY, ARISING OUT OF OR IN CONNECTION WITH THE IMPLEMENTATION, USE, COMMERCIALIZATION OR PERFORMANCE OF THIS INTELLECTUAL PROPERTY.

This license is effective until terminated. You may terminate it at any time by destroying the Intellectual Property together with all copies in any form. The license will also terminate if you fail to comply with any term or condition of this Agreement. Except as provided in the following sentence, no such termination of this license shall require the termination of any third party end-user sublicense to the Intellectual Property which is in force as of the date of notice of such termination. In addition, should the Intellectual Property, or the operation of the Intellectual Property, infringe, or in LICENSOR's sole opinion be likely to infringe, any patent, copyright, trademark or other right of a third party, you agree that LICENSOR, in its sole discretion, may terminate this license without any compensation or liability to you, your licensees or any other party. You agree upon termination of any kind to destroy or cause to be destroyed the Intellectual Property together with all copies in any form, whether held by you or by any third party.

Except as contained in this notice, the name of LICENSOR or of any other holder of a copyright in all or part of the Intellectual Property shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Intellectual Property without prior written authorization of LICENSOR or such copyright holder. LICENSOR is and shall at all times be the sole entity that may authorize you or any third party to use certification marks, trademarks or other special designations to indicate compliance with any LICENSOR standards or specifications.

This Agreement is governed by the laws of the Commonwealth of Massachusetts. The application to this Agreement of the United Nations Convention on Contracts for the International Sale of Goods is hereby expressly excluded. In the event any provision of this Agreement shall be deemed unenforceable, void or invalid, such provision shall be modified so as to make it valid and enforceable, and as so modified the entire Agreement shall remain in full force and effect. No decision, action or inaction by LICENSOR shall be construed to be a waiver of any rights or remedies available to it.

None of the Intellectual Property or underlying information or technology may be downloaded or otherwise exported or reexported in violation of U.S. export laws and regulations. In addition, you are responsible for complying with any local laws in your jurisdiction which may impact your right to import, export or use the

Intellectual Property, and you represent that you have complied with any regulations or registration procedures required by applicable law to make this license enforceable.

Table of Contents

i. Preface . . . 8
ii. Submitting organizations . . . 9
iii. Submission contact points . . . 9
iv. Revision history . . . 10
Major changes between versions 1.0 and 1.1 . . . 11
v. Changes to the OGC® Abstract Specification . . . 13
Foreword . . . 14
Introduction . . . 15
RDF . . . 15
SPARQL . . . 15
GeoSPARQL Standard structure . . . 16
OGC GeoSPARQL – A Geographic Query Language for RDF Data . . . 19
1. Scope . . . 20
2. Conformance . . . 21
3. Normative references . . . 23
4. Terms and definitions . . . 24
5. Conventions . . . 25
5.1. Symbols and abbreviated terms . . . 25
5.2. Namespaces . . . 25
5.3. Placeholder IRIs . . . 26
5.4. RDF Serializations . . . 26
6. Core . . . 27
6.1. SPARQL . . . 27
6.2. Classes . . . 27
6.3. Standard Properties for geo:SpatialObject . . . 28
6.4. Standard Properties for geo:Feature . . . 28
6.5. Standard Properties for geo:SpatialMeasure . . . 33
7. Topology Vocabulary Extension (relation_family) . . . 34
7.1. Parameters . . . 34
7.2. Simple Features Relation Family (relation_family=Simple Features) . . . 34
7.3. Egenhofer Relation Family (relation_family=Egenhofer) . . . 35
7.4. RCC8 Relation Family (relation_family=RCC8) . . . 36
7.5. Equivalent RCC8, Egenhofer and Simple Features Topological Relations . . . 37
8. Geometry Extension (serialization, version) . . . 38
8.1. Parameters . . . 38
8.2. Geometry Class . . . 39
8.3. Standard Properties for geo:Geometry . . . 39
8.4. Geometry Serializations . . . 42

8.5. Non-topological Query Functions . . . 50
9. Geometry Topology Extension (relation_family, serialization, version) . . . 56
9.1. Parameters . . . 56
9.2. Common Query Functions . . . 56
9.3. Simple Features Relation Family (relation_family=Simple Features) . . . 57
9.4. Egenhofer Relation Family (relation_family=Egenhofer) . . . 58
9.5. Requirements for RCC8 Relation Family (relation_family=RCC8) . . . 58
10. RDFS Entailment Extension (relation_family, serialization, version) . . . 60
10.1. Parameters . . . 60
10.2. Common Requirements . . . 60
10.3. WKT Serialization (serialization=WKT) . . . 60
10.4. GML Serialization (serialization=GML) . . . 61
11. Query Rewrite Extension (relation_family, serialization, version) . . . 62
11.1. Parameters . . . 63
11.2. Simple Features Relation Family (relation_family=Simple Features) . . . 63
11.3. Egenhofer Relation Family (relation_family=Egenhofer) . . . 63
11.4. RCC8 Relation Family (relation_family=RCC8) . . . 64
11.5. Special Considerations . . . 65
12. Future Work . . . 66
Annex A (normative) Abstract Test Suite . . . 67
A.1 Conformance Class: *Core* . . . 68
A.1.1 `/conf/core/sparql-protocol` . . . 68
A.1.2 `/conf/core/spatial-object-class` . . . 68
A.1.3 `/conf/core/spatial-measure-class` . . . 68
A.1.3 `/conf/core/feature-class` . . . 69
A.2 Conformance Class: Topology Vocabulary Extension (relation_family) . . . 70
A.2.1 relation_family = Simple Features . . . 70
A.2.2 relation_family = Egenhofer . . . 70
A.2.3 relation_family = RCC8 . . . 70
A.3 Conformance Class: Geometry Extension (serialization, version) . . . 72
A.3.1 Tests for all Serializations . . . 72
A.3.2 serialization = WKT . . . 73
A.3.3 serialization = GML . . . 75
A.3.4 serialization = GEOJSON . . . 76
A.3.5 serialization = KML . . . 77
A.3.6 serialization = DGGS . . . 79
A.4 Conformance Class: Geometry Topology Extension (relation_family, serialization, version) . . . 81
A.4.1 Tests for all relation families . . . 81
A.4.2 relation_family = Simple Features . . . 81
A.4.3 relation_family = Egenhofer . . . 81
A.4.4 relation_family = RCC8 . . . 82

A.5 Conformance Class: RDFS Entailment Extension (relation_family, serialization, version) 83
A.5.1 Tests for all implementations 83
A.5.2 serialization=WKT 83
A.5.3 serialization=GML 83
A.6 Conformance Class: Query Rewrite Extension (relation_family, serialization, version) 85
A.6.1 relation_family = Simple Features 85
A.6.2 relation_family = Egenhofer 85
A.6.3 relation_family = RCC8 86
Annex B (informative) GeoSPARQL Examples 87
B.1 RDF Examples 88
B.1.1 Classes 88
B.1.2 Properties 93
B.2 Example SPARQL Queries & Rules 100
B.2.1 Example Data 100
B.2.2 Example Queries 102
B.2.3 Example Rule Application 105
B.2.4 Example Geometry Serialization Conversion Functions 107
Bibliography 108

i. Preface

The GeoSPARQL standard defines:

- a formal *profile*
- **this specification document**
  - a core RDF/OWL ontology for geographic information representation
- a set of SPARQL extension functions
- a Functions & Rules vocabulary, derived from the ontology
- a Simple Features feature types vocabulary
- a set of RIF rules, and
- SHACL shapes for RDF data validation

This document has the role of *specification* and authoratitivly defines many of the standard's elements, including the ontology, SPRQL functions and function and rule vocabularies. Complete descriptions of the standard's parts and their roles are given in the Introduction in the section GeoSPARQL Standard structure.

ii. Submitting organizations

The following organizations submitted this Implementation Specification to the Open Geospatial Consortium Inc.:

a. Australian Bureau of Meteorology
b. Bentley Systems, Inc.
c. CSIRO
d. Defence Geospatial Information Working Group (DGIWG)
e. GeoConnections - Natural Resources Canada
f. Interactive Instruments GmbH
g. GeoScape Australia
h. Mainz University Of Applied Sciences
i. Oracle America
j. Ordnance Survey
k. Raytheon Company
l. SURROUND Australia Pty Ltd.
m. Traverse Technologies, Inc.
n. US Geological Survey (USGS)

iii. Submission contact points

All questions regarding this submission should be directed to the editor or the submitters:

Contact	Company
Matthew Perry	Oracle America
John Herring	Oracle America
Nicholas J. Car	SURROUND Australia Pty Ltd.
Timo Homburg	Mainz University Of Applied Sciences
Simon J.D. Cox	CSIRO

iv. Revision history

Date	Release	Author	Paragraph modified	Description
27 Oct. 2009	Draft	Matthew Perry	Clause 6	Technical Draft
11 Nov. 2009	Draft	John R. Herring	All	Creation
06 Jan. 2010	Draft	John R. Herring	All	Comment responses
30 March 2010	Draft	Matthew Perry	All	Comment responses
26 Oct. 2010	Draft	Matthew Perry	All	Revision based on working group discussion
28 Jan. 2011	Draft	Matthew Perry	All	Revision based on working group discussion
18 April 2011	Draft	Matthew Perry	All	Restructure with multiple conformance classes
02 May 2011	Draft	Matthew Perry	Clause 6 and Clause 8	Move Geometry Class from core to geometryExtension
05 May 2011	Draft	Matthew Perry	All	Update URIs
13 Jan. 2012	Draft	Matthew Perry	All	Revision based on Public RFC
16 April 2012	Draft	Matthew Perry	All	Revision based on adoption vote comments
19 July 2012	1.0	Matthew Perry	All	Revision of URIs based on OGC Naming Authority recommendations
09 Oct. 2020	1.1 Draft	Joseph Abhayaratna	All	Establishment of the 1.1 Specification

Date	Release	Author	Paragraph modified	Description
10 Oct. 2020 to 15 March 2021	1.1 Draft	GeoSPARQL 1.1 SWG	All	Addition of GeoSPARQL 1.1 elements

Major changes between versions 1.0 and 1.1

Version 1.1 of GeoSPARQL was released approximately 9 years after version 1.0. It contains no breaking changes to 1.0, but does contain additions: whole new profile resources, new ontology elements and new functions. The major changes are given in the tables below.

These new profile resources are resources - documents - that are separate from this specification. The new *profile defintion* lists all the GeoSPARQL 1.1 resources.

New resource	Location
Profile definition	http://www.opengis.net/def/geosparql
GeoSPARQL Rules in RIF	http://www.opengis.net/def/geosparql-rifrules
RDF validation file	http://www.opengis.net/def/geosparql-shapes

These new ontology elements and new functions are normatively defined in this specification document.

New element	Section
Classes	
Spatial Measure class	Section 6.2.3
Feature Properties	
hasBoundingBox	Section 6.4.3
hasCentroid	Section 6.4.4
hasLength	Section 6.4.5
hasArea	Section 6.4.6
hasVolume	Section 6.4.7
Geometry Properties	
inSRS	Section 8.3.8
Geometry Serializations	
asWKT function	Section 8.4.1.3
asGML function	Section 8.4.2.3
geoJSONLiteral	Section 8.4.3.1

New element	Section
asGeoJSON	Section 8.4.3.2
asGeoJSON function	Section 8.4.3.3
kmlLiteral	Section 8.4.4.1
asKML	Section 8.4.4.2
asKML function	Section 8.4.4.3
dggsLiteral	Section 8.4.5.1
asDGGSWKT	[Property: geo:asDGGSWKT]
asDGGSWKT function	[Function: geo:asDGGSWKT]
Non-topological Query Functions	
maxX	Section 8.5.11
maxY	Section 8.5.12
maxZ	Section 8.5.13
minX	Section 8.5.14
minY	Section 8.5.15
minZ	Section 8.5.16
Spatial Aggregate Functions	
BBOX	[Function: geosaf:BBOX]
BoundingCircle	[Function: geoaf:BoundingCircle]
Centroid	[Function: geoaf:Centroid]
ConcatLines	[Function: geoaf:ConcatLines]
ConcaveHull	[Function: geoaf:ConcaveHull]
ConvexHull	[Function: geoaf:ConvexHull]
Union	[Function: geoaf:Union]

v. Changes to the OGC® Abstract Specification

The OGC® Abstract Specification does not require changes to accommodate this OGC® standard.

Foreword

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights. However, to date, no such rights have been claimed or identified.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the specification set forth in this document, and to provide supporting documentation.

Introduction

The W3C Semantic Web Activity is defining a collection of technologies that enables a “web of data” where information is easily shared and reused across applications. Some key pieces of this technology stack are the RDF (Resource Description Framework) data model [RDF], [RDFS], the OWL Web Ontology Language [OWL2] and the SPARQL protocol and RDF query language [SPARQL].

RDF

RDF is, among other things, a data model built on edge-node "graphs." Each link in a graph consists of three things (with many aliases depending on the mapping from other types of data models):

- Subject (start node, instance, entity, feature)
- Predicate (verb, property, attribute, relation, member, link, reference)
- Object (value, end node, non-literal values can be used as a Subject)

Any of the three values in a triple can be represented with a Internationalized Resource Identifier (IRI) [IETF3987], which globally and uniqueliy identifies the resource referenced. IRIs are an extension to Universal Resource Identifiers (URIs) that allow for non-ASCII characters. In addition to functioning as identifiers, IRIs are usually, but not necissarily, resolvable which means a person or machine can "dereference" them (*click on them* or otherwise action them) and be taken to more information about the resource, perhaps in a web browser.

Subjects and objects within an RDF triple are called nodes and can also be be represented with a blank node (a local identifier with meaning outside the graph it is defined within). Objects can further be represented with a literal value. Basic literal values in RDF are those used in XML [XSD2] but the basic types can be extended for specialised purposes and in this specification are, for geometry data.

Note that the same node may be a subject in some triples, and tan object in others.

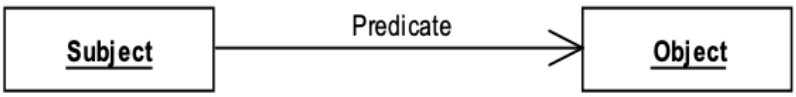

Figure 1. RDF Triple

Almost all data can be presented or represented in RDF. In particular, it is an easy match to the (feature-instance-by-id, attribute, value) tuples of the General Feature Model [ISO19109], and for the relational model as (table primary key, column, value).

SPARQL

From [SPARQL]:

> SPARQL ... is a set of specifications that provide languages and protocols to query and manipulate RDF graph content on the Web or in an RDF store.

and, from Wikipedia[1]:

> SPARQL (pronounced "sparkle" /□sp□□k□l/, a recursive acronym for SPARQL Protocol and RDF Query Language) is an RDF query language - that is, a semantic query language for databases — able to retrieve and manipulate data stored in Resource Description Framework (RDF) format. It was made a standard by the RDF Data Access Working Group (DAWG) of the World Wide Web Consortium, and is recognized as one of the key technologies of the semantic web. On 15 January 2008, SPARQL 1.0 was acknowledged by W3C as an official recommendation, and SPARQL 1.1 in March, 2013.

SPARQL queries work on RDF representations of data by finding patterns that match templates in the query, in effect finding information graphs in the RDF data based on the templates and filters (constraints on nodes and edges) expressed in the query. This query template is represented in the SPARQL query by a set of parameterized “query variables” appearing in a sequence of RDF triples and filters. If the query processor finds a set of triples in the data (converted to an RDF graph in some predetermined standard manner) then the values that the “query variables” take on in those triples become a solution to the query request. The values of the variables are returned in the query result in a format based on the “SELECT” clause of the query (similar to SQL).

In addition to predicates defined in this manner, the SPARQL query may contain filter functions that can be used to further constrain the query. Several mechanisms are available to extend filter functions to allow for predicates calculated directly on data values. The SPARQL specification [SPARQL] in section 17.6[2] describes the mechanism for invocation of such a filter function.

The OGC GeoSPARQL standard supports representing and querying geospatial data on the Semantic Web. GeoSPARQL defines a vocabulary for representing geospatial data in RDF, and it defines extensions to the SPARQL query language for processing geospatial data.

GeoSPARQL Standard structure

The GeoSPARQL standard comprises multiple parts:

- a formal *profile*
  - http://www.opengis.net/def/geosparql
  - defined according to the *Profiles Vocabulary* [PROF]
  - this relates the parts in the standard together, provides access to them, and declares dependencies on other standards
- **this specification document**
  - which defines many of the standard’s parts
- a core RDF/OWL [RDF],[OWL2] ontology for geographic information representation
  - based on the General Feature Model [ISO19109], Simple Features [ISO19125-1], Feature Geometry [ISO19107] and SQL MM [ISO13249]

    - defined within the specification document and delivered in RDF also
- a Functions & Rules vocabulary, derived from the ontology
    - presented as a [SKOS] taxonomy
- a Simple Features feature types vocabulary
    - presented as a [SKOS] taxonomy
- a set of SPARQL [SPARQL] extension functions
    - defined within the specification document
- a set of RIF [RIFCORE] rules, and
    - templated within the specification document
    - also delivered as a RIF document also
- SHACL [SHACL] shapes for RDF data validation
    - defined within a shapes graph file

This specification document follows further modular design; it comprises several different components:

- a *core* component defining the top-level RDFS/OWL classes for spatial objects
- a *topology vocabulary* component defining the RDF properties for asserting and querying topological relations between spatial objects
- a *geometry* component defines RDFS data types for serializing geometry data, geometry-related RDF properties, and non-topological spatial query functions for geometry objects
- a *geometry topology* component defining topological query functions
- an *RDFS entailment* component defining mechanisms for matching implicit RDF triples that are derived based on RDF and RDFS semantics
- a *query rewrite* component defining rules for transforming a simple triple pattern that tests a topological relation between two features into an equivalent query involving concrete geometries and topological query functions

Each of these specification components forms a *requirements class* for GeoSPARQL. Implementations can provide various levels of functionality by choosing which requirements classes to support. For example, a system based purely on qualitative spatial reasoning may support only the core and topological vocabulary components.

In addition, GeoSPARQL is designed to accommodate systems based on qualitative spatial reasoning and systems based on quantitative spatial computations. Systems based on qualitative spatial reasoning, (e.g. those based on the Region Connection Calculus [QUAL], [LOGIC]) do not usually model explicit geometries, so queries in such systems will likely test for binary spatial relationships between features rather than between explicit geometries. To allow queries for spatial relations between features in quantitative systems, GeoSPARQL defines a series of query transformation rules that expand a feature-only query into a geometry-based query. With these transformation rules, queries about spatial relations between features will have the same specification in both qualitative systems and quantitative systems. The qualitative system will likely evaluate the query

with a backward-chaining spatial “reasoner”, and the quantitative system can transform the query into a geometry-based query that can be evaluated with computational geometry.

[1] https://en.wikipedia.org/wiki/SPARQL

[2] https://www.w3.org/TR/sparql11-query/#extensionFunctions

OGC GeoSPARQL – A Geographic Query Language for RDF Data

1. Scope

This is the specification document for GeoSPARQL which, as a whole, comprises multiple parts. See the Introduction section GeoSPARQL Standard structure for details of the parts.

GeoSPARQL does not define a comprehensive vocabulary for representing spatial information. It instead defines a core set of classes, properties and datatypes that can be used to construct query patterns. Many useful extensions to this vocabulary are possible, and we intend for the Semantic Web and Geographic Information System (GIS) communities to develop additional vocabulary for describing spatial information.

2. Conformance

Conformance with this specification shall be checked using all the relevant tests specified in Annex A (normative). The framework, concepts, and methodology for testing, and the criteria to be achieved to claim conformance are specified in *ISO 19105: Geographic information — Conformance and Testing* [ISO19105].

This document establishes several requirements classes and corresponding conformance classes. Any GeoSPARQL implementation claiming conformance with one of the conformance classes shall pass all the tests in the associated abstract test suite. Requirements and conformance test IRIs defined in this document are relative to the namespace IRI `http://www.opengis.net/spec/geosparql/1.1/`. Table 1 summarizes the conformance classes in GeoSPARQL. Many conformance classes are parameterized. For parameterized conformance classes, the list of parameters is given within parenthesis.

Table 1. Conformance Classes

Conformance class	Description	Subclause of the abstract test suite
Core	Defines top-level spatial vocabulary components	A.1
Topology Vocabulary Extension (relation_family)	Defines topological relation vocabulary	A.2
Geometry Extension (serialization, version)	Defines geometry vocabulary and non- topological query functions	A.3
Geometry Topology Extension (serialization, version, relation_family)	Defines topological query functions for geometry objects	A.4
RDFS Entailment Extension (serialization, version , relation_family)	Defines a mechanism for matching implicit RDF triples that are derived based on RDF and RDFS semantics	A.5
Query Rewrite Extension (serialization, version, relation_family)	Defines query transformation rules for computing spatial relations between spatial objects based on their associated geometries	A.6

Dependencies between each GeoSPARQL requirements class are shown below in Figure 2. To support a requirements class for a given set of parameter values, an implementation must support each dependent requirements class with the same set of parameter values.

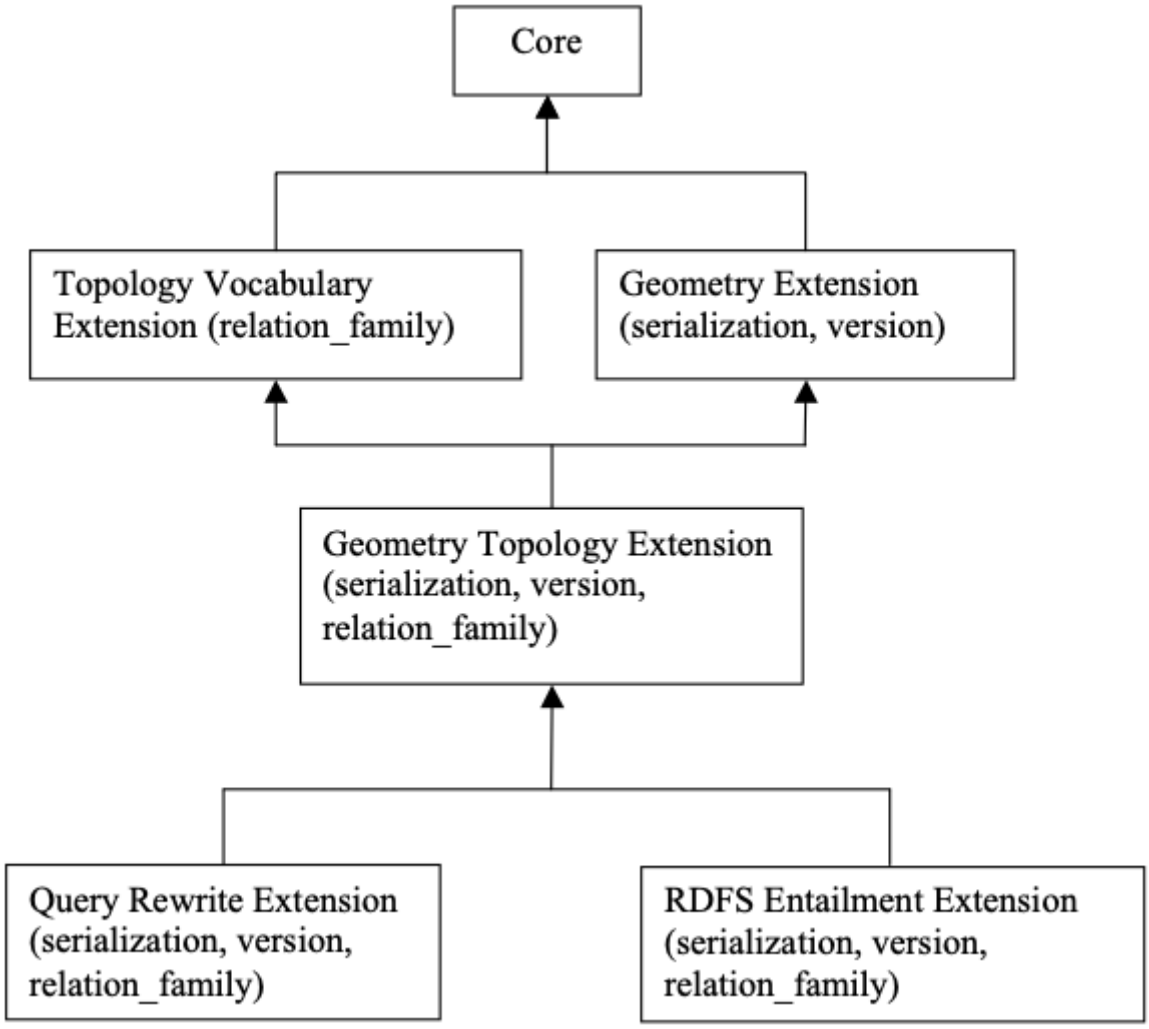

Figure 2. Requirements Class Dependency Graph

3. Normative references

The following normative documents contain provisions which, through reference in this text, constitute provisions of this document. For dated references, subsequent amendments to, or revisions of, any of these publications do not apply. However, parties to agreements based on this document are encouraged to investigate the possibility of applying the most recent editions of the normative documents indicated below. For undated references, the latest edition of the normative document referred to applies.

- [ISO19125-1], *ISO 19125-1: Geographic information — Simple feature access — Part 1: Common architecture*
- [OGC07-036], *OGC 07-036: Geography Markup Language (GML) Encoding Standard*, Version 3.2.1 (27 August 2007).
- [IETF3987], Internet Engineering Task Force, *RFC 3987: Internationalized Resource Identifiers (IRIs)*. IETF Request for Comment (January 2005). https://tools.ietf.org/html/rfc3987
- [RIFCORE], *RIF Core Dialect (Second Edition)*, W3C Recommendation (5 February 2013) http://www.w3.org/TR/rif-core/
- [SPARQL], *SPARQL 1.1 Query Language*, W3C Recommendation (21 March 2013). https://www.w3.org/TR/sparql11-query/
- [SPARQLENT], *SPARQL 1.1 Entailment Regimes*, W3C Recommendation (21 March 2013). https://www.w3.org/TR/sparql11-entailment/
- [SPARQLPROT], *SPARQL 1.1 Protocol*, W3C Recommendation (21 March 2013)> http://www.w3.org/TR/sparql11-protocol/
- [SPARQLRESX], *SPARQL Query Results XML Format (Second Edition)*, W3C Recommendation (21 March 2013). https://www.w3.org/TR/rdf-sparql-XMLres/
- [SPARQLRESJ], *SPARQL 1.1 Query Results JSON Format*, W3C Recommendation (21 March 2013). http://www.w3.org/TR/sparql11-results-json/

4. Terms and definitions

For the purposes of this document, the terms and definitions given in the above references apply.

5. Conventions

5.1. Symbols and abbreviated terms

In this specification, the following common acronyms are used:

CRS	Coordinate Reference System
DGGS	Discrete Global Grid System
GeoJSON	Geographic JavaScript Object Notation
GFM	General Feature Model (as defined in ISO 19109)
GML	Geography Markup Language
KML	Keyhole Markup Language
OWL	OWL 2 Web Ontology Language
RCC	Region Connection Calculus
RDF	Resource Description Framework
RDFS	RDF Schema
RIF	Rule Interchange Format
SPARQL	SPARQL Protocol and RDF Query Language
SRS	Spatial Reference System
WKT	Well Known Text (as defined by Simple Features or ISO 19125)
W3C	World Wide Web Consortium (http://www.w3.org/)
XML	Extensible Markup Language

5.2. Namespaces

The following IRI namespace prefixes are used throughout this document:

ogc:	http://www.opengis.net/
eg:	http://example.com/
geo:	http://www.opengis.net/ont/geosparql#
geof:	http://www.opengis.net/def/function/geosparql/
geor:	http://www.opengis.net/def/rule/geosparql/
sf:	http://www.opengis.net/ont/sf#
skos:	http://www.w3.org/2004/02/skos/core#
gml:	http://www.opengis.net/ont/gml#
my:	http://example.org/ApplicationSchema#
xsd:	http://www.w3.org/2001/XMLSchema#

rdf:	http://www.w3.org/1999/02/22-rdf-syntax-ns#
rdfs:	http://www.w3.org/2000/01/rdf-schema#
owl:	http://www.w3.org/2002/07/owl#

5.3. Placeholder IRIs

The IRI `ogc:geomLiteral` is used in requirement specifications as a placeholder for the geometry literal serialization used in a fully-qualified conformance class, e.g. `<http://www.opengis.net/ont/geosparql#wktLiteral>`. The IRI `ogc:asGeomLiteral` is used in requirement specifications as a placeholder for the geometry literal serialization property used in a fully-qualified conformance class, e.g. `geo:asWKT`.

5.4. RDF Serializations

Three RDF serializations are used in this document. Terse RDF Triple Language (turtle) [TURTLE] is used for RDF snippets placed within the main body of the document, and turtle, JSON-LD [JSON-LD] & RDF/XML [RDFXML] is used for the examples in Annex B.

6. Core

This clause establishes the **core** requirements class, with IRI `/req/core`, which has a single corresponding conformance class, **core**, with IRI `/conf/core`. This requirements class defines a set of classes and properties for representing geospatial data. The resulting vocabulary can be used to construct SPARQL graph patterns for querying appropriately modeled geospatial data. RDFS and OWL vocabulary have both been used so that the vocabulary can be understood by systems that support only RDFS entailment and by systems that support OWL-based reasoning.

6.1. SPARQL

Req 1 Implementations shall support the SPARQL Query Language for RDF [SPARQL], the SPARQL Protocol [SPARQLPROT] and the SPARQL Query Results XML [SPARQLRESX] and JSON [SPARQLRESJ] Formats.
`/req/core/sparql-protocol`

6.2. Classes

Three main classes are defined: `geo:SpatialObject`, `geo:SpatialMeasure` and `geo:Feature`. The class `geo:Feature` is equivalent to the UML class `Feature` defined in [ISO19109].

6.2.1. Class: geo:SpatialObject

The class `geo:SpatialObject` is defined by the following:

```
geo:SpatialObject a rdfs:Class, owl:Class ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "Spatial Object"@en ;
    skos:definition "The class Spatial Object represents everything that can
                     have a spatial representation. It is superclass of feature
                     and geometry"@en .
```

Req 2 Implementations shall allow the RDFS class `geo:SpatialObject` to be used in SPARQL graph patterns.
`/req/core/spatial-object-class`

Example:

```
eg:x a geo:SpatialObject ;
     skos:prefLabel "Object X";
.
```

6.2.2. Class: geo:Feature

The class `geo:Feature` is equivalent to the class `GFI_Feature` [ISO19156] and is defined by the following:

```
geo:Feature a rdfs:Class, owl:Class ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "Feature"@en ;
    rdfs:subClassOf geo:SpatialObject ;
    owl:disjointWith geo:Geometry ;
    skos:definition "This class represents the top-level feature type. This
                     class is equivalent to GFI_Feature defined in ISO 19156,
                     and it is superclass of all feature types."@en .
```

Req 4 Implementations shall allow the RDFS class `geo:Feature` to be used in SPARQL graph patterns.
`/req/core/feature-class`

6.2.3. Class: geo:SpatialMeasure

The class `geo:SpatialMeasure` is defined by the following:

```
geo:SpatialMeasure a rdfs:Class, owl:Class ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "Spatial Measure"@en ;
    skos:definition "This class represents a measurement of some dimension
                     of a feature's spatial presence."@en .
```

NOTE	Properties for Spatial Measure may need to indicate a use a unit of measure. When they do, OGC recommended units of measure vocabularies should be used. See the OGC Definitions Server[3].

Req 3 Implementations shall allow the RDFS class `geo:SpatialMeasure` to be used in SPARQL graph patterns.
`/req/core/spatial-measure-class`

6.3. Standard Properties for geo:SpatialObject

To be defined

6.4. Standard Properties for geo:Feature

Properties are defined for associating geometries with features.

Req 9 Implementations shall allow the properties `geo:hasGeometry`, `geo:hasDefaultGeometry`, `geo:hasLength`, `geo:hasArea`, `geo:hasVolume` `geo:hasCentroid`, `geo:hasBoundingBox` and `geo:hasSpatialResolution` to be used in SPARQL graph patterns.
`/req/geometry-extension/feature-properties`

6.4.1. Property: geo:hasGeometry

The property `geo:hasGeometry` is used to link a feature with a geometry that represents its spatial extent. A given feature may have many associated geometries.

```
geo:hasGeometry a rdf:Property, owl:ObjectProperty ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "has Geometry"@en ;
    skos:definition "A spatial representation for a given feature."@en ;
    rdfs:domain geo:Feature;
    rdfs:range geo:Geometry .
```

6.4.2. Property: geo:hasDefaultGeometry

The property `geo:hasDefaultGeometry` is used to link a feature with its default geometry. The default geometry is the geometry that should be used for spatial calculations in the absence of a request for a specific geometry (e.g. in the case of query rewrite).

```
geo:hasDefaultGeometry a rdf:Property, owl:ObjectProperty ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "has Default Geometry"@en ;
    skos:definition "The default geometry to be used in spatial calculations,
                 usually the most detailed geometry."@en ;
    rdfs:subPropertyOf geo:hasGeometry;
    rdfs:domain geo:Feature;
    rdfs:range geo:Geometry .
```

GeoSPARQL does not restrict the cardinality of the `geo:hasDefaultGeometry` property. It is thus possible for a feature to have more than one distinct default geometry or to have no default geometry. This situation does not result in a query processing error; SPARQL graph pattern matching simply proceeds as normal. Certain queries may, however, give logically inconsistent results. For example, if a feature `my:f1` has two asserted default geometries, and those two geometries are disjoint polygons, the query below could return a non-zero count on a system supporting the GeoSPARQL Query Rewrite Extension (rule `geor:sfDisjoint`).

```
PREFIX geo: <http://www.opengis.net/ont/geosparql#>

SELECT (COUNT(*) AS ?cnt)
WHERE { :f1 geo:sfDisjoint :f1 }
```

Such cases are application-specific data modeling errors and are therefore outside of the scope of the GeoSPARQL specification., however it is recommended that multiple geometries indicated with `geo:hasDefaultGeometry` should be differentiated by `Geometry` class properties, perhaps relating to precision, SRS etc.

6.4.3. Property: geo:hasBoundingBox

The property `geo:hasBoundingBox` is used to link a feature with a simplified geometry-representation corresponding to the envelope of its geometry. Bounding-boxes are typically uses in indexing and discovery.

```
geo:hasBoundingBox a rdf:Property, owl:ObjectProperty ;
    rdfs:subPropertyOf geo:hasGeometry;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "has bounding box"@en ;
    skos:definition "The minimum or smallest bounding or enclosing box of a given
feature."@en ;
    skos:scopeNote "The target is a geometry that defines a rectilinear region whose
edges are aligned with the axes of the coordinate reference system, which exactly
contains the geometry or feature e.g. sf:Envelope"@en ;
    rdfs:domain geo:Feature ;
    rdfs:range geo:Geometry .
```

GeoSPARQL does not restrict the cardinality of the `geo:hasBoundingBox` property. A feature may be associated with more than one bounding-box, for example in different coordinate reference systems.

6.4.4. Property: geo:hasCentroid

The property `geo:hasCentroid` is used to link a feature with a point geometry corresponding with the centroid of its geometry. The centroid is typically used to show location on a low-resolution image, and for some indexing and discovery functions.

```
geo:hasCentroid a rdf:Property, owl:ObjectProperty ;
    rdfs:subPropertyOf geo:hasGeometry;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "has centroid"@en ;
    skos:definition "The arithmetic mean position of all the geometry points of a
given feature."@en ;
    skos:scopeNote "The target geometry shall describe a point, e.g. sf:Point"@en ;
    rdfs:domain geo:Feature ;
    rdfs:range geo:Geometry .
```

GeoSPARQL does not restrict the cardinality of the `geo:hasCentroid` property. A feature may be associated with more than one centroid, for example computed using different rules or in different coordinate reference systems.

6.4.5. Property: geo:hasLength

The property `geo:hasLength` is used to indicate the length of a `geo:Feature`. In the case of a one-dimensional `geo:Feature`, it is the simple length. In the case of a two-dimensional `geo:Feature`, it is interpreted to mean the perimeter length. The range of the property is `geo:SpatialMeasure`, which encodes the length value expressed as a scalar quantity which also includes the units of measure, and potentially uncertainty and other properties.

```
geo:hasLength a rdf:Property, owl:ObjectProperty ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "has length"@en ;
    skos:definition "The length of a Feature, expressed as a Spatial Measure."@en ;
    rdfs:domain geo:Feature ;
    rdfs:range geo:SpatialMeasure .
```

TIP

A consistency check can be applied to geometries indicating both this property and the `geo:dimension` property: if supplied, the `geo:dimension` property's range value must be the literal integer 1 or 2. The following SPARQL query will return `true` if applied to a graph where is not always the case for all geometries:

```
    PREFIX geo: <http://www.opengis.net/ont/geosparql#>
    ASK
    WHERE {
        ?g geo:hasLength ?l ;
           geo:dimension ?d .

        FILTER (?d > 2)
    }
```

6.4.6. Property: geo:hasArea

The property `geo:hasArea` is used to indicate the area of a `geo:Feature`. The range of the property is `geo:SpatialMeasure`, which encodes the area value expressed as a scalar quantity which also includes the units of measure, and potentially uncertainty and other properties.

```
geo:hasArea a rdf:Property, owl:ObjectProperty;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "has area"@en ;
    skos:definition "The two-dimensional area of a Feature, expressed as a Spatial
Measure."@en ;
    rdfs:domain geo:Feature ;
    rdfs:range geo:SpatialMeasure .
```

TIP

A consistency check can be applied to geometries indicating both this property and the `geo:dimension` property: if supplied, the `geo:dimension` property's range value must be the literal integer 2. The following SPARQL query will return `true` if applied to a graph where is not always the case for all geometries:

```
PREFIX geo: <http://www.opengis.net/ont/geosparql#>
ASK
WHERE {
    ?g geo:hasArea ?a ;
       geo:dimension ?d .

    FILTER (?d != 2)
}
```

6.4.7. Property: geo:hasVolume

The property `geo:hasVolume` is used to indicate the volume of a `geo:Feature`. The range of the property is `geo:SpatialMeasure`, which encodes the volume value expressed as a scalar quantity which also includes the units of measure, and potentially uncertainty and other properties.

```
geo:hasVolume a rdf:Property, owl:ObjectProperty;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "has volume"@en ;
    skos:definition "The volume of a Feature, expressed as a
                     Spatial Measure"@en ;
    rdfs:domain geo:Feature ;
    rdfs:range geo:SpatialMeasure .
```

TIP

A consistency check can be applied to geometries indicating both this property and the `geo:dimension` property: if supplied, the `geo:dimension` property's range value must be the literal integer 3. The following SPARQL query will return `true` if applied to a graph where is not always the case for all geometries:

```
PREFIX geo: <http://www.opengis.net/ont/geosparql#>
ASK
WHERE {
    ?g geo:hasVolume ?a ;
       geo:dimension ?d .

    FILTER (?d != 3)
}
```

6.5. Standard Properties for geo:SpatialMeasure

To be defined

[3] https://www.ogc.org/def-server

7. Topology Vocabulary Extension (relation_family)

This clause establishes the *Topology Vocabulary Extension (relation_family)* parameterized requirements class, with IRI `/req/topology-vocab-extension`, which has a single corresponding conformance class *Topology Vocabulary Extension (relation_family)*, with IRI `/conf/topology-vocab-extension`. This requirements class defines a vocabulary for asserting and querying topological relations between spatial objects. The class is parameterized so that different families of topological relations may be used, e.g. RCC8, Egenhofer. These relations are generalized so that they may connect features as well as geometries.

A Dimensionally Extended 9-Intersection Model (DE-9IM) pattern, which specifies the spatial dimension of the intersections of the interiors, boundaries and exteriors of two geometric objects, is used to describe each spatial relation. Possible pattern values are `-1 (empty)`, `0, 1, 2, T (true) = {0, 1, 2}`, `F (false) = {-1}`, `* (don't care) = {-1, 0, 1, 2}`. In the following descriptions, the notation `X/Y` is used denote applying a spatial relation to geometry types `X` and `Y` (i.e., `x` *relation* `y` where `x` is of type `X` and `y` is of type `Y`). The symbol `P` is used for 0- dimensional geometries (e.g. points). The symbol `L` is used for 1-dimensional geometries (e.g. lines), and the symbol `A` is used for 2-dimensional geometries (e.g. polygons). Consult the Simple Features specification [ISO19125-1] for a more detailed description of DE-9IM intersection patterns.

7.1. Parameters

The following parameter is defined for the *Topology Vocabulary Extension* requirements class.

relation_family: Specifies the set of topological spatial relations to support.

7.2. Simple Features Relation Family (relation_family=Simple Features)

This clause defines requirements for the *Simple Features* relation family.

Req 5 Implementations shall allow the properties `geo:sfEquals`, `geo:sfDisjoint`, `geo:sfIntersects`, `geo:sfTouches`, `geo:sfCrosses`, `geo:sfWithin`, `geo:sfContains`, `geo:sfOverlaps` to be used in SPARQL graph patterns.
`/req/topology-vocab-extension/sf-spatial-relations`

Topological relations in the *Simple Features* family are summarized in Table 2. Multi-row intersection patterns should be interpreted as a logical `OR` of each row.

Table 2. Simple Features Topological Relations

Relation Name	Relation IRI	Domain/Range	Applies To Geometry Types	DE-9IM Intersection Pattern
equals	`geo:sfEquals`	`geo:SpatialObject`	`All`	`(TFFFTFFFT)`

Relation Name	Relation IRI	Domain/Range	Applies To Geometry Types	DE-9IM Intersection Pattern
disjoint	`geo:sfDisjoint`	`geo:SpatialObject`	`All`	`(FF**FF****)`
intersects	`geo:sfIntersects`	`geo:SpatialObject`	`All`	`(T******** *T******* ***T***** ****T****)`
touches	`geo:sfTouches`	`geo:SpatialObject`	`All except P/P`	`(FT******* F**T***** F***T****)`
within	`geo:sfWithin`	`geo:SpatialObject`	`All`	`(T*F**F***)`
contains	`geo:sfContains`	`geo:SpatialObject`	`All`	`(T*****FF*)`
overlaps	`geo:sfOverlaps`	`geo:SpatialObject`	`A/A, P/P, L/L`	`(T*T***T**) for A/A, P/P`; `(1*T***T**) for L/L`
crosses	`geo:sfCrosses`	`geo:SpatialObject`	`P/L, P/A, L/A, L/L`	`(T*T***T**) for P/L, P/A, L/A`; `(0********) for L/L`

7.3. Egenhofer Relation Family (relation_family=Egenhofer)

This clause defines requirements for the 9-intersection model for binary topological relations (*Egenhofer*) relation family. Consult references [FORMAL] and [CATEG] for a more detailed discussion of *Egenhofer* relations.

Req 6 Implementations shall allow the properties `geo:ehEquals`, `geo:ehDisjoint`, `geo:ehMeet`, `geo:ehOverlap`, `geo:ehCovers`, `geo:ehCoveredBy`, `geo:ehInside`, `geo:ehContains` to be used in SPARQL graph patterns.
`/req/topology-vocab-extension/eh-spatial-relations`

Topological relations in the *Egenhofer* family are summarized in Table 3. Multi-row intersection patterns should be interpreted as a logical `OR` of each row.

Table 3. Egenhofer Topological Relations

Relation Name	Relation IRI	Domain/Range	Applies To Geometry Types	DE-9IM Intersection Pattern
equals	`geo:ehEquals`	`geo:SpatialObject`	`All`	`(TFFFTFFFT)`
disjoint	`geo:ehDisjoint`	`geo:SpatialObject`	`All`	`(FF*FF****)`

Relation Name	Relation IRI	Domain/Range	Applies To Geometry Types	DE-9IM Intersection Pattern
meet	`geo:ehMeet`	`geo:SpatialObject`	`All except P/P`	`(FT******* F**T***** F***T****)`
overlap	`geo:ehOverlap`	`geo:SpatialObject`	`All`	`(T*T***T**)`
covers	`geo:ehCovers`	`geo:SpatialObject`	`A/A, A/L, L/L`	`(T*TFT*FF*)`
covered by	`geo:ehCoveredBy`	`geo:SpatialObject`	`A/A, L/A, L/L`	`(TFF*TFT**)`
inside	`geo:ehInside`	`geo:SpatialObject`	All	`(TFF*FFT**)`
contains	`geo:ehContains`	`geo:SpatialObject`	All	`(T*TFF*FF*)`

7.4. RCC8 Relation Family (relation_family=RCC8)

This clause defines requirements for the region connection calculus basic 8 (*RCC8*) relation family. Consult references [QUAL] and [LOGIC] for a more detailed discussion of *RCC8* relations.

Req 7 Implementations shall allow the properties `geo:rcc8eq`, `geo:rcc8dc`, `geo:rcc8ec`, `geo:rcc8po`, `geo:rcc8tppi`, `geo:rcc8tpp`, `geo:rcc8ntpp`, `geo:rcc8ntppi` to be used in SPARQL graph patterns.
`/req/topology-vocab-extension/rcc8-spatial-relations`

Topological relations in the *RCC8* family are summarized in Table 4.

Table 4. RCC8 Topological Relations

Relation Name	Relation IRI	Domain/Range	Applies To Geometry Types	DE-9IM Intersection Pattern
equals	`geo:rcc8eq`	`geo:SpatialObject`	`A/A`	`(TFFFTFFFT)`
disconnected	`geo:rcc8dc`	`geo:SpatialObject`	`A/A`	`(FFTFFTTTT)`
externally connected	`geo:rcc8ec`	`geo:SpatialObject`	`A/A`	`(FFTFTTTTT)`
partially overlapping	`geo:rcc8po`	`geo:SpatialObject`	`A/A`	`(TTTTTTTTT)`
tangential proper part inverse	`geo:rcc8tppi`	`geo:SpatialObject`	`A/A`	`(TTTFTTFFT)`
tangential proper part	`geo:rcc8tpp`	`geo:SpatialObject`	`A/A`	`(TFFTTFTTT)`
non-tangential proper part	`geo:rcc8ntpp`	`geo:SpatialObject`	`A/A`	`(TFFTFFTTT)`
non-tangential proper part inverse	`geo:rcc8ntppi`	`geo:SpatialObject`	`A/A`	`(TTTFFTFFT)`

7.5. Equivalent RCC8, Egenhofer and Simple Features Topological Relations

Table 5 summarizes the equivalences between *Egenhofer*, *RCC8* and *Simple Features* spatial relations for closed, non-empty regions. The symbol + denotes logical OR, and the symbol ¬ denotes negation.

Table 5. Equivalent Simple Features, RCC8 and Egenhofer relations

Simple Features	**RCC8**	**Egenhofer**
equals	equals	equals
disjoint	disconnected	disjoint
intersects	¬ disconnected	¬ disjoint
touches	externally connected	meet
within	non-tangential proper part + tangential proper part	inside + coveredBy
contains	non-tangential proper part inverse + tangential proper part inverse	contains + covers
overlaps	partially overlapping	overlap

8. Geometry Extension (serialization, version)

This clause establishes the *Geometry Extension (serialization, version)* parameterized requirements class, with IRI `/req/geometry-extension`, which has a single corresponding conformance class *Geometry Extension (serialization, version)*, with IRI `/conf/geometry-extension`. This requirements class defines a vocabulary for asserting and querying information about geometry data, and it defines query functions for operating on geometry data.

As part of the vocabulary, an RDFS datatype is defined for encoding detailed geometry information as a literal value. A literal representation of a geometry is needed so that geometric values may be treated as a single unit. Such a representation allows geometries to be passed to external functions for computations and to be returned from a query.

Other schemes for encoding simple geometry data in RDF have been implemented. The W3C Basic Geo vocabulary[4] was an early (2003) RDF vocabulary for "representing lat(itude), long(itude) and other information about spatially-located things, using WGS84 as a reference datum" and many widely used Semantic Web vocabularies contain some spatial data support. For example, *Dublin Core Terms* provides a *Location* class[5] for "A spatial region or named place." and *schema.org* provides a number of spatial object and geometry classes, such as `GeoCoordinates`[6] and `GeoShape`[7].

Many vocabularies, such as these two, provide little specific support for detailed geometries and only support the WGS84 Coordinate Reference System (CSR).

Since 2012 and the first version of GeoSPARQL, many ontologies have imported GeoSPARQL, for example, the _ISA Programme Location Core Vocabulary_[8] whose usage notes provide examples containing GeoSPARQL literals and the use of GeoSPARQL's "geometry class". The W3C's more recent DCAT2 standard[9] similarly contains usage notes for `geometry`, `bbox` and other properties that suggest the use of GeoSPARQL literals.

Some of the properties defined in these vocabularies, such as `hasSpatialResolution` which are introduced in this version of GeoSPARQL, were motivated by similar properties introduced in external vocabularies such as DCAT2, which provided some of the motivation for their inclusion here. The GeoSPARQL 1.1 Standards Working Group charter [CHARTER] contains references to a number of vocabularies/ontologies that were influential in the generation of this verison of GeoSPARQL.

8.1. Parameters

The following parameters are defined for the *Geometry Extension* requirements class.

serialization

Specifies the serialization standard to use when generating geometry literals and also the supported geometry types.

Note that the serialization chosen strongly affects the geometry conceptualization. The WKT

serialization aligns the geometry types with *ISO 19125 Simple Features* [ISO19125-1], and the GML serialization aligns the geometry types with *ISO 19107 Spatial Schema* [ISO19107].

version

Specifies the version of the serialization format used.

8.2. Geometry Class

A single root geometry class is defined: `geo:Geometry`. The class `geo:Geometry` is equivalent to the UML class `GM_Object` defined in [ISO19107]. In addition, properties are defined for describing geometry data and for associating geometries with features.

8.2.1. Class: geo:Geometry

The class `geo:Geometry` is equivalent to `GM_Object` [ISO19107] and is defined by the following:

```
geo:Geometry a rdfs:Class, owl:Class ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "Geometry"@en ;
    rdfs:subClassOf geo:SpatialObject ;
    owl:disjointWith geo:Feature;
    skos:definition "The class represents the top-level geometry type. This class
                     is equivalent to the UML class GM_Object defined in ISO 19107,
                     and it is superclass of all geometry types."@en .
```

Req 8 Implementations shall allow the RDFS class `geo:Geometry` to be used in SPARQL graph patterns.
`/req/geometry-extension/geometry-class`

8.3. Standard Properties for geo:Geometry

Properties are defined for describing geometry metadata.

Req 10 Implementations shall allow the properties `geo:dimension`, `geo:coordinateDimension`, `geo:spatialDimension`, `geo:isEmpty`, `geo:isSimple`, `geo:hasSerialization` , `geo:inSRS` to be used in SPARQL graph patterns.
`/req/geometry-extension/geometry-properties`

8.3.1. Property: geo:dimension

The dimension is the topological dimension of this geometric object, which must be less than or equal to the coordinate dimension. In non-homogeneous collections, this will return the largest topological dimension of the contained objects.

```
geo:dimension a rdf:Property, owl:DatatypeProperty ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "dimension"@en ;
    skos:definition "The topological dimension of this geometric object, which
                     must be less than or equal to the coordinate dimension. In
                     non-homogeneous collections, this is the largest
                     topological dimension of the contained objects."@en ;
    rdfs:domain geo:Geometry ;
    rdfs:range xsd:integer .
```

8.3.2. Property: geo:coordinateDimension

The coordinate dimension is the dimension of direct positions (coordinate tuples) used in the definition of this geometric object.

```
geo:coordinateDimension a rdf:Property, owl:DatatypeProperty;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "coordinate dimension"@en ;
    skos:definition "The number of measurements or axes needed to describe the
                     position of this geometry in a coordinate system."@en ;
    rdfs:domain geo:Geometry ;
    rdfs:range xsd:integer .
```

8.3.3. Property: geo:spatialDimension

The spatial dimension is the dimension of the spatial portion of the direct positions (coordinate tuples) used in the definition of this geometric object. If the direct positions do not carry a measure coordinate, this will be equal to the coordinate dimension.

```
geo:spatialDimension a rdf:Property, owl:DatatypeProperty;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "spatial dimension"@en ;
    skos:definition "The number of measurements or axes needed to describe the
                     spatial position of this geometry in a coordinate system."@en ;
    rdfs:domain geo:Geometry ;
    rdfs:range xsd:integer .
```

8.3.4. Property: geo:hasSpatialResolution

The property `geo:hasSpatialResolution` is used to indicate resolution of the elements within literal representations of a geometry. Since this property is defined for a `geo:Geometry`, all literal representations of that geometry must have the same spatial resolution.

```
geo:hasSpatialResolution a rdf:Property, owl:ObjectProperty;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "has spatial resolution"@en ;
    skos:definition "The spatial resolution of a Geometry"@en ;
    rdfs:domain geo:Geometry .
```

8.3.5. Property: geo:isEmpty

The `geo:isEmpty` Boolean will be set to `true` only if the geometry contains no information.

```
geo:isEmpty a rdf:Property, owl:DatatypeProperty ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "is empty"@en ;
    skos:definition "(true) if this geometric object is the empty Geometry. If
                    true, then this geometric object represents the empty point
                    set for the coordinate space."@en ;
    rdfs:domain geo:Geometry ;
    rdfs:range xsd:boolean .
```

8.3.6. Property: geo:isSimple

The `geo:isSimple` Boolean will be set to `true`, only if the geometry contains no self-intersections, with the possible exception of its boundary.

```
geo:isSimple a rdf:Property, owl:DatatypeProperty ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "is simple"@en ;
    skos:definition "(true) if this geometric object has no anomalous geometric
                    points, such as self intersection or self tangency."@en ;
    rdfs:domain geo:Geometry ;
    rdfs:range xsd:boolean .
```

8.3.7. Property: geo:hasSerialization

The `geo:hasSerialization` property is used to connect a geometry with its text-based serialization (e.g., its WKT serialization).

```
geo:hasSerialization a rdf:Property, owl:DatatypeProperty ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "has serialization"@en ;
    skos:definition "Connects a geometry object with its text-based serialization."@en
;
    rdfs:domain geo:Geometry ;
    rdfs:range rdfs:Literal .
```

NOTE	this property is the generic property used to connect a geometry with its serialization. GeoSPARQL also contains a number of sub properties of this one for connecting serializations of common types with geometries, for example `geo:asGeoJSON` which can be used for GeoJSON [GEOJSON] literals.

8.3.8. Property: geo:inSRS

The `geo:inSRS` property is used to connect a geometry with the SRS used for its representation which affects measurements of its size (length, area, volume).

```
geo:inSRS a rdf:Property, owl:ObjectProperty ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "in SRS"@en ;
    skos:definition "The spatial reference system used for the literal representation
of the geometry."@en ;
    rdfs:domain geo:Geometry ;
    rdfs:range skos:Concept .
```

8.4. Geometry Serializations

This section establishes the requirements for representing geometry data in RDF based on different systems.

8.4.1. Well-Known Text (serialization=WKT)

This section establishes the requirements for representing geometry data in RDF based on Well-Known Text (WKT) as defined by Simple Features [ISO19125-1]. It defines one RDFS Datatype: `http://www.opengis.net/ont/geosparql#wktLiteral` and one property, `http://www.opengis.net/ont/geosparql#asWKT`.

8.4.1.1. RDFS Datatype: geo:wktLiteral

```
geo:wktLiteral a rdfs:Datatype ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "Well-known Text literal"@en ;
    skos:definition "A Well-known Text serialization of a geometry object."@en .
```

Req 11 All RDFS Literals of type `geo:wktLiteral` shall consist of an optional IRI identifying the coordinate reference system and a required Well Known Text (WKT) description of a geometric value. Valid `geo:wktLiterals` are formed by either a WKT string as defined in [ISO13249] or by concatenating a valid absolute IRI, as defined in [IETF3987], enclose in angled brackets (< & >) followed by a single space (Unicode U+0020 character) as a separator, and a WKT string as defined in [ISO13249].
`/req/geometry-extension/wkt-literal`

The following *ABNF* [IETF5234] syntax specification formally defines this literal:

```
wktLiteral ::= opt-iri-and-space geometric-data

opt-iri-and-space = "<" IRI ">" LWSP / ""
```

The token `opt-iri-and-space` may be either an IRI and space or nothing (`""`), the token `IRI` (Internationalized Resource Identifier) is essentially a web address and is defined in [IETF3987] and the token `LWSP`, is one or more white space characters, as defined in [IETF5234]. `geometric-data` is the Well-Known Text representation of the geometry, defined in [ISO13249].

In the absence of a leading spatial reference system IRI, the following spatial reference system IRI will be assumed: `<http://www.opengis.net/def/crs/OGC/1.3/CRS84>`. This IRI denotes WGS 84 longitude-latitude.

Req 12 The IRI `<http://www.opengis.net/def/crs/OGC/1.3/CRS84>` shall be assumed as the spatial reference system for `geo:wktLiteral` instances that do not specify an explicit spatial reference system IRI.
`/req/geometry-extension/wkt-literal-default-srs`

The OGC maintains a set of SRS IRIs under the `http://www.opengis.net/def/crs/` namespace and IRIs from this set are recommended for use, however others may also be used, as long as they are valid IRIs.

Req 13 Coordinate tuples within `geo:wktLiteral` shall be interpreted using the axis order defined in the spatial reference system used.
`/req/geometry-extension/wkt-axis-order`

The example `geo:wktLiteral` below encodes a point geometry using the default WGS84 geodetic longitude-latitude spatial reference system:

```
"Point(-83.38 33.95)"^^<http://www.opengis.net/ont/geosparql#wktLiteral>
```

A second example below encodes the same point as encoded in the example above but using a SRS identified by `http://www.opengis.net/def/SRS/EPSG/0/4326`: a WGS 84 geodetic latitude-longitude spatial reference system (note that this spatial reference system defines a different axis order):

```
"<http://www.opengis.net/def/crs/EPSG/0/4326> Point(33.95
-83.38)"^^<http://www.opengis.net/ont/geosparql#wktLiteral>
```

Req 14 An empty RDFS Literal of type `geo:wktLiteral` shall be interpreted as an empty geometry.
`/req/geometry-extension/wkt-literal-empty`

8.4.1.2. Property: geo:asWKT

The `geo:asWKT` property is defined to link a geometry with its WKT serialization.

Req 15 Implementations shall allow the RDF property `geo:asWKT` to be used in SPARQL graph patterns.
`/req/geometry-extension/geometry-as-wkt-literal`

The property `geo:asWKT` is used to link a geometric element with its WKT serialization.

```
geo:asWKT a rdf:Property, owl:DatatypeProperty ;
    rdfs:subPropertyOf geo:hasSerialization ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "as WKT"@en ;
    skos:definition "The WKT serialization of a geometry."@en ;
    rdfs:domain geo:Geometry ;
    rdfs:range geo:wktLiteral .
```

8.4.1.3. Function: geof:asWKT

```
geof:asWKT (geom: ogc:geomLiteral): geo:wktLiteral
```

The function `geof:asWKT` converts `geom` to an equivalent WKT representation preserving the coordinate reference system.

Req 15.x Implementations shall support `geof:asWKT` as a SPARQL extension function.
`/req/geometry-extension/asWKT-function`

8.4.2. Geography Markup Language (serialization=GML)

This section establishes requirements for representing geometry data in RDF based on GML as defined by Geography Markup Language Encoding Standard [OGC07-036]. It defines one RDFS Datatype: `http://www.opengis.net/ont/geosparql#gmlLiteral` and one property, `http://www.opengis.net/ont/geosparql#asGML`.

8.4.2.1. RDFS Datatype: geo:gmlLiteral

```
geo:gmlLiteral a rdfs:Datatype ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "GML literal"@en ;
    skos:definition "The datatype of GML literal values"@en .
```

Valid `geo:gmlLiteral` instances are formed by encoding geometry information as a valid element from the GML schema that implements a subtype of `GM_Object`. For example, in GML 3.2.1 this is every element directly or indirectly in the substitution group of the element `{http://www.opengis.net/ont/gml/3.2}AbstractGeometry`. In GML 3.1.1 and GML 2.1.2 this is every

element directly or indirectly in the substitution group of the element `{http://www.opengis.net/ont/gml}_Geometry`.

Req 16 All `geo:gmlLiteral` instances shall consist of a valid element from the GML schema that implements a subtype of `GM_Object` as defined in [OGC07-036].
`/req/geometry-extension/gml-literal`

The example `geo:gmlLiteral` below encodes a point geometry in the WGS 84 geodetic longitude-latitude spatial reference system using GML version 3.2:

```
"""
<gml:Point
        srsName=\"http://www.opengis.net/def/crs/OGC/1.3/CRS84\"
        xmlns:gml=\"http://www.opengis.net/ont/gml\">
    <gml:pos>-83.38 33.95</gml:pos>
</gml:Point>
"""^^<http://www.opengis.net/ont/geosparql#gmlLiteral>
```

Req 17 An empty `geo:gmlLiteral` shall be interpreted as an empty geometry.
`/req/geometry-extension/gml-literal-empty`

Req 18 Implementations shall document supported GML profiles.
`/req/geometry-extension/gml-profile`

8.4.2.2. Property: geo:asGML

This document defines the `geo:asGML` property to link a geometry with its serialization.

Req 19 Implementations shall allow the RDF property `geo:asGML` to be used in SPARQL graph patterns.
`/req/geometry-extension/geometry-as-gml-literal`

The property `geo:asGML` is used to link a geometric element with its GML serialization.

```
geo:asGML a rdf:Property ;
    rdfs:subPropertyOf geo:hasSerialization ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "as GML"@en ;
    skos:definition "The GML serialization of a geometry."@en ;
    rdfs:domain geo:Geometry ;
    rdfs:range geo:gmlLiteral .
```

8.4.2.3. Function: geof:asGML

```
geof:asGML (geom: ogc:geomLiteral, gmlProfile: xsd:string): geo:gmlLiteral
```

The function `geof:asGML` converts `geom` to an equivalent GML representation defined by a gmlProfile version string preserving the coordinate reference system.

Req 19.x Implementations shall support `geof:asGML` as a SPARQL extension function.
`/req/geometry-extension/asGML-function`

8.4.3. GeoJSON (serialization=GEOJSON)

This section establishes requirements for representing geometry data in RDF based on GeoJSON as defined by [GeoJSON]. It defines one RDFS Datatype: `http://www.opengis.net/ont/geosparql#geoJSONLiteral` and one property, `http://www.opengis.net/ont/geosparql#asGeoJSON`.

8.4.3.1. RDFS Datatype: geo:geoJSONLiteral

```
geo:geoJSONLiteral a rdfs:Datatype ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "GeoJSON Literal"@en ;
    skos:definition "A GeoJSON serialization of a geometry object."@en .
```

Valid `geo:geoJSONLiteral` instances are formed by encoding geometry information as a Geometry object as defined in the GeoJSON specification [GEOJSON].

Req 20 All `geo:geoJSONLiteral` instances shall consist of the Geometry objects as defined in the GeoJSON specification [GEOJSON].
`/req/geometry-extension/geojson-literal`

Req 21 RDFS Literals of type `geo:geoJSONLiteral` do not contain a SRS definition. All literals of this type shall, according to the GeoJSON specification, be encoded only in, and be assumed to use, the WGS84 geodetic longitude-latitude spatial reference system (`http://www.opengis.net/def/crs/OGC/1.3/CRS84`).
`/req/geometry-extension/geojson-literal-srs`

The example `geo:geoJSONLiteral` below encodes a point geometry using the default WGS84 geodetic longitude-latitude spatial reference system for Simple Features 1.0:

```
"""
{"type": "Point", "coordinates": [-83.38,33.95]}
"""^^<http://www.opengis.net/ont/geosparql#geoJSONLiteral>
```

Req 22 An empty RDFS Literal of type `geo:geoJSONLiteral` shall be interpreted as an empty geometry, i.e. `{"geometry": null}` in GeoJSON .
`/req/geometry-extension/geojson-literal-empty`

8.4.3.2. Property: geo:asGeoJSON

The `geo:asGeoJSON` property is defined to link a geometry with its GeoJSON serialization.

Req 23 Implementations shall allow the RDF property `geo:asGeoJSON` to be used in SPARQL graph patterns.
`/req/geometry-extension/geometry-as-geojson-literal`

The property `geo:asGeoJSON` is used to link a geometric element with its GeoJSON serialization.

```
geo:asGeoJSON a rdf:Property, owl:DatatypeProperty ;
    rdfs:subPropertyOf geo:hasSerialization ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "as GeoJSON"@en ;
    skos:definition "The GeoJSON serialization of a geometry."@en ;
    rdfs:domain geo:Geometry ;
    rdfs:range geo:geoJSONLiteral .
```

8.4.3.3. Function: geof:asGeoJSON

```
geof:asGeoJSON (geom: ogc:geomLiteral): geo:geoJSONLiteral
```

The function `geof:asGeoJSON` converts `geom` to an equivalent GeoJSON representation. Coordinates are converted to the CRS84 coordinate system, the only valid coordinate system to be used in a GeoJSON literal.

Req 23.x Implementations shall support `geof:asGeoJSON` as a SPARQL extension function.
`/req/geometry-extension/asGeoJSON-function`

8.4.4. Keyhole Markup Language (serialization=KML)

This section establishes requirements for representing geometry data in RDF based on KML as defined by [OGCKML]. It defines one RDFS Datatype: `http://www.opengis.net/ont/geosparql#kmlLiteral` and one property, `http://www.opengis.net/ont/geosparql#asKML`.

8.4.4.1. RDFS Datatype: geo:kmlLiteral

```
geo:kmlLiteral a rdfs:Datatype ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "KML Literal"@en ;
    skos:definition "A KML serialization of a geometry object."@en .
```

Valid `geo:kmlLiteral` instances are formed by encoding geometry information as a Geometry object as defined in the KML specification [OGCKML].

Req 24 All `geo:kmlLiteral` instances shall consist of the Geometry objects as defined in the KML specification [OGCKML].
`/req/geometry-extension/kml-literal`

Req 25 RDFS Literals of type `geo:kmlLiteral` do not contain a SRS definition. All literals of this type shall according to the KML specification only be encoded in and assumed to use the WGS84 geodetic longitude-latitude spatial reference system (`http://www.opengis.net/def/crs/OGC/1.3/CRS84`).
`/req/geometry-extension/kml-literal-srs`

The example `geo:kmlLiteral` below encodes a point geometry using the default WGS84 geodetic longitude-latitude spatial reference system for Simple Features 1.0:

```
"""
<Point xmlns=\"http://www.opengis.net/kml/2.2\">
    <coordinates>-83.38,33.95</coordinates>
</Point>
"""^^<http://www.opengis.net/ont/geosparql#kmlLiteral>
```

Req 26 An empty RDFS Literal of type `geo:kmlLiteral` shall be interpreted as an empty geometry .
`/req/geometry-extension/kml-literal-empty`

8.4.4.2. Property: geo:asKML

The `geo:asKML` property is defined to link a geometry with its KML serialization.

Req 27 Implementations shall allow the RDF property `geo:asKML` to be used in SPARQL graph patterns.
`/req/geometry-extension/geometry-as-kml-literal`

The property `geo:asKML` is used to link a geometric element with its KML serialization.

```
geo:asKML a rdf:Property, owl:DatatypeProperty;
    rdfs:subPropertyOf geo:hasSerialization ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "as KML"@en ;
    skos:definition "The KML serialization of a geometry."@en ;
    rdfs:domain geo:Geometry ;
    rdfs:range geo:kmlLiteral .
```

8.4.4.3. Function: geof:asKML

```
geof:asKML (geom: ogc:geomLiteral): geo:kmlLiteral
```

The function `geof:asKML` converts `geom` to an equivalent KML representation. Coordinates are

converted to the CRS84 coordinate system, the only valid coordinate system to be used in a KML literal.

Req 27.x Implementations shall support `geof:asKML` as a SPARQL extension function.
`/req/geometry-extension/asKML-function`

8.4.5. Discrete Global Grid System (serialization=DGGS)

This section establishes the requirements for representing geometry data in RDF as represented in a Discrete Global Grid System (DGGS), in text. The form of representation is known as a *DGGS Well-Known Text* geometry representation and is based on elements of the second version of the DGGS *Abstract Specification* [DGGSAS]. It defines one RDFS Datatype: `http://www.opengis.net/ont/geosparql#dggsLiteral` and one property, `http://www.opengis.net/ont/geosparql#asDGGS`.

8.4.5.1. RDFS Datatype: geo:dggsLiteral

```
geo:dggsLiteral a rdfs:Datatype ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "DGGS Well-Known Text Literal"@en ;
    skos:definition "A textual serialization of a Discrete Global Grid (DGGS) geometry
object."@en .
```

Valid `geo:dggsLiteral` instances are formed by encoding geometry information as text and as required by a particular DGGS and in accordance with the *Discrete Global Grid System Abstract Specification* [DGGSAS]. An indication of the particular DGGS, as well as the geometric information must also be indicated in the literal as per the following *ABNF* [IETF5234] syntax specification:

```
dggsLiteral ::= "<" IRI ">" LWSP geometric-data
```

The token `IRI` (Internationalized Resource Identifier) is essentially a web address and is defined in [IETF3987] and the token `LWSP`, is one or more white space characters, as defined in [IETF5234]. `geometric-data` is potentially specific to the DGGS and is not specified here.

Req 28 All RDFS Literals of type `geo:dggsLiteral` shall consist of a required DGGS identifier, an IRI, and a DGGS geometry serialization formulated according to the identified DGGS.
`/req/geometry-extension/dggswkt-literal`

The example `geo:dggsLiteral` below encodes a point geometry according to the *AusPIX* DGGS[10]. The DGGS geometry type is indicated with the token `OrdinateList` and the point, enclosed in parenthesis, is identified with the AusPIX-specific 'Cell ID' of *R3234*:

```
"<https://w3id.org/dggs/auspix> OrdinateList
(R3234)"^^<http://www.opengis.net/ont/geosparql#dggsLiteral>
```

Req 29 An empty RDFS Literal of type `geo:dggsLiteral` shall be interpreted as an empty geometry.
`/req/geometry-extension/dggs-literal-empty`

8.4.5.2. Property: geo:asDGGS

The `geo:asDGGS` property is defined to link a geometry with its DGGS serialization.

Req 30 Implementations shall allow the RDF property `geo:asDGGS` to be used in SPARQL graph patterns.
`/req/geometry-extension/geometry-as-dggs-literal`

The property `geo:asDGGS` is used to link a Geometry instance with its serialization.

```
geo:asDGGS a rdf:Property, owl:DatatypeProperty ;
    rdfs:subPropertyOf geo:hasSerialization ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "as DGGS"@en ;
    skos:definition "The DGGS Well-Known Text serialization of a geometry."@en ;
    rdfs:domain geo:Geometry ;
    rdfs:range geo:dggsLiteral .
```

8.4.5.3. Function: geof:asDGGS

```
geof:asDGGS (geom: ogc:geomLiteral, dggsIri: xsd:anyURI): geo:DggsLiteral
```

The function `geof:asDGGS` converts `geom` to an equivalent DGGS representation.

Req 15.x Implementations shall support `geof:asDGGS` as a SPARQL extension function.
`/req/geometry-extension/asDGGS-function`

8.5. Non-topological Query Functions

This clause defines SPARQL functions for performing non-topological spatial operations.

Req 31 Implementations shall support `geof:distance`, `geof:buffer`, `geof:convexHull`, `geof:intersection`, `geof:union`, `geof:difference`, `geof:symDifference`, `geof:envelope`, `geof:boundary` as SPARQL extension functions, consistent with the definitions of their corresponding functions (`distance`, `buffer`, `convexHull`, `intersection`, `difference`, `symDifference`, `envelope` and `boundary` respectively) in Simple Features [ISO19125-1] and other attached definitions respectively.
`/req/geometry-extension/query-functions`

An invocation of any of the following functions with invalid arguments produces an error. An invalid argument includes any of the following:

- An argument of an unexpected type

- An invalid geometry literal value
- A geometry literal from a spatial reference system that is incompatible with the spatial reference system used for calculations
- An invalid units IRI

For further discussion of the effects of errors during FILTER evaluation, consult Section 17[11] of the SPARQL specification [SPARQL].

Note that returning values instead of raising an error serves as an extension mechanism of SPARQL.

From Section 17.3.1[12] of the SPARQL specification [SPARQL]:

> SPARQL language extensions may provide additional associations between operators and operator functions; ... No additional operator may yield a result that replaces any result other ... . The consequence of this rule is that SPARQL `FILTER` s will produce at least the same intermediate bindings after applying a `FILTER` as an unextended implementation.

This extension mechanism enables GeoSPARQL implementations to simultaneously support multiple geometry serializations. For example, a system that supports `geo:wktLiteral` serializations may also support `geo:gmlLiteral` serializations and consequently would not raise an error if it encounters multiple geometry datatypes while processing a given query.

NOTE	Several non-topological query functions use a unit of measure IRI. The OGC has recommended units of measure vocabularies for use, see the OGC Definitions Server[13].

8.5.1. Function: geof:distance

```
geof:distance (geom1: ogc:geomLiteral,
               geom2: ogc:geomLiteral,
               units: xsd:anyURI): xsd:double
```

Returns the shortest distance between any two Points in the two geometric objects. Calculations are in spatial reference system of `geom1`.

8.5.2. Function: geof:buffer

```
geof:buffer (geom: ogc:geomLiteral,
             radius: xsd:double,
             units: xsd:anyURI): ogc:geomLiteral
```

Returns a geometric object that represents all Points whose distance from `geom1` is less than or equal to the `radius` measured in `units`. Calculations are in the spatial reference system of `geom1`.

8.5.3. Function: geof:convexHull

```
geof:convexHull (geom1: ogc:geomLiteral): ogc:geomLiteral
```

Returns a geometric object that represents all Points in the convex hull of `geom1`. Calculations are in the spatial reference system of `geom1`.

8.5.4. Function: geof:intersection

```
geof:intersection (geom1: ogc:geomLiteral,
                   geom2: ogc:geomLiteral): ogc:geomLiteral
```

Returns a geometric object that represents all Points in the intersection of `geom1` with `geom2`. Calculations are in the spatial reference system of `geom1`.

8.5.5. Function: geof:union

```
geof:union (geom1: ogc:geomLiteral,
            geom2: ogc:geomLiteral): ogc:geomLiteral
```

This function returns a geometric object that represents all Points in the union of `geom1` with `geom2`. Calculations are in the spatial reference system of `geom1`.

8.5.6. Function: geof:difference

```
geof:difference (geom1: ogc:geomLiteral,
                 geom2: ogc:geomLiteral): ogc:geomLiteral
```

This function returns a geometric object that represents all Points in the set difference of `geom1` with `geom2`. Calculations are in the spatial reference system of `geom1`.

8.5.7. Function: geof:symDifference

```
geof:symDifference (geom1: ogc:geomLiteral,
                    geom2: ogc:geomLiteral): ogc:geomLiteral
```

This function returns a geometric object that represents all Points in the set symmetric difference of `geom1` with `geom2`. Calculations are in the spatial reference system of `geom1`.

8.5.8. Function: geof:envelope

```
geof:envelope (geom1: ogc:geomLiteral): ogc:geomLiteral
```

This function returns the minimum bounding box of `geom1`. Calculations are in the spatial reference system of `geom1`.

8.5.9. Function: geof:boundary

```
geof:boundary (geom1: ogc:geomLiteral): ogc:geomLiteral
```

This function returns the closure of the boundary of `geom1`. Calculations are in the spatial reference system of `geom1`.

8.5.10. Function: geof:getSRID

```
geof:getSRID (geom: ogc:geomLiteral): xsd:anyURI
```

Returns the spatial reference system IRI for `geom`.

Req 32 Implementations shall support `geof:getSRID`, `geof:concaveHull`, `geof:minBoundingCircle`, `geof:union2`, `geof:concatLines`, `geof:centroid`, `geof:maxX`, `geof:maxY`, `geof:maxZ`, `geof:minX`, `geof:minY` and `geof:minZ` as a SPARQL extension functions.
`/req/geometry-extension/srid-function`

8.5.11. Function: geof:maxX

```
geof:maxX (geom: ogc:geomLiteral): xsd:double
```

The function `geof:maxX` returns the maximum X coordinate for `geom`.

8.5.12. Function: geof:maxY

```
geof:maxY (geom: ogc:geomLiteral): xsd:double
```

The function `geof:maxY` returns the maximum Y coordinate for `geom`.

8.5.13. Function: geof:maxZ

```
geof:maxZ (geom: ogc:geomLiteral): xsd:double
```

The function `geof:maxZ` returns the maximum Z coordinate for `geom`.

8.5.14. Function: geof:minX

```
geof:minX (geom: ogc:geomLiteral): xsd:double
```

The function `geof:minX` returns the minimum X coordinate for `geom`.

8.5.15. Function: geof:minY

```
geof:minY (geom: ogc:geomLiteral): xsd:double
```

The function `geof:minY` returns the minimum Y coordinate for `geom`.

8.5.16. Function: geof:minZ

```
geof:minZ (geom: ogc:geomLiteral): xsd:double
```

The function `geof:minZ` returns the minimum Z coordinate for `geom`.

8.5.17. Function: geof:transform

```
geof:transform (geom: ogc:geomLiteral, srsIRI: xsd:anyURI): ogc:geomLiteral
```

geof:transform converts `geom` to a spatial reference system defined by srsIRI. The function raises an error if a transformation is not mathematically possible.

NOTE	We recommend that implementers use the same literal type as a result of this function that is passed as a parameter to this function.

8.5.18. Function: geof:minBoundingCircle

```
geof:boundingCircle (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function `geof:boundingCircle` calculates a minimum bounding circle of the set of given geometries.

8.5.19. Function: geof:centroid

```
geof:centroid (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function `geof:centroid` valculates the centroid of the set of given geometries.

8.5.20. Function: geof:concatLines

```
geof:concatLines (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function `geof:concatLines` Concatenates a set of LineStrings.

8.5.21. Function: geof:concaveHull

```
geof:concaveHull (geom: ogc:geomLiteral, targetPercent: xsd:double): ogc:geomLiteral
```

The function `geof:concaveHull` calculates the concave hull of the set of given geometries.

8.5.22. Function: geof:union2

```
geof:union2 (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function `geof:union2` calculates the union of the set of given geometries.

[4] http://www.w3.org/2003/01/geo/

[5] http://purl.org/dc/terms/Location

[6] https://schema.org/GeoCoordinates

[7] https://schema.org/GeoShape

[8] https://www.w3.org/ns/locn

[9] https://www.w3.org/TR/vocab-dcat/#spatial-properties

[10] https://w3id.org/dggs/auspix - this is a semi-formal identifier for AusPIX. Likley, in time, a more official identifier regime for DGGSes will emerge, similar to the OGC's SRS register

[11] https://www.w3.org/TR/sparql11-query/#expressions

[12] https://www.w3.org/TR/sparql11-query/#operatorExtensibility

[13] https://www.ogc.org/def-server

9. Geometry Topology Extension (relation_family, serialization, version)

This clause establishes the *Geometry Topology Extension (relation_family, serialization, version)* parameterized requirements class, with IRI `/req/geometry-topology-extension`, which defines a collection of topological query functions that operate on geometry literals. This class is parameterized to give implementations flexibility in the topological relation families and geometry serializations that they choose to support. This requirements class has a single corresponding conformance class *Geometry Topology Extension (relation_family, serialization, version)*, with IRI `/conf/geometry-topology-extension`.

The Dimensionally Extended Nine Intersection Model (DE-9IM) [DE-9IM] has been used to define the relation tested by the query functions introduced in this section. Each query function is associated with a defining DE-9IM intersection pattern. Possible pattern values are:

- `-1` (empty)
- `0`, `1`, `2`, `T` (true) = `{0, 1, 2}`
- `F` (false) = `{-1}`
- `*` (don't care) = `{-1, 0, 1, 2}`

In the following descriptions, the notation `X/Y` is used denote applying a spatial relation to geometry types `X` and `Y` (i.e., `x` *relation* `y` where `x` is of type `X` and `y` is of type `Y`). The symbol `P` is used for 0-dimensional geometries (e.g. points). The symbol `L` is used for 1- dimensional geometries (e.g. lines), and the symbol `A` is used for 2-dimensional geometries (e.g. polygons). Consult the Simple Features specification [ISO19125-1] for a more detailed description of DE-9IM intersection patterns.

9.1. Parameters

- **relation_family**: Specifies the set of topological spatial relations to support.
- **serialization**: Specifies the serialization standard to use for geometry literals.
- **version**: Specifies the version of the serialization format used.

9.2. Common Query Functions

Req 34 Implementations shall support `geof:relate` as a SPARQL extension function, consistent with the relate operator defined in Simple Features [ISO19125-1].
`/req/geometry-topology-extension/relate-query-function`

```
geof:relate (geom1: ogc:geomLiteral,
             geom2: ogc:geomLiteral,
             pattern-matrix: xsd:string): xsd:boolean
```

Returns `true` if the spatial relationship between `geom1` and `geom2` corresponds to one with acceptable

values for the specified pattern-matrix. Otherwise, this function returns `false`. `pattern-matrix` represents a DE-9IM intersection pattern consisting of `T` (true) and `F` (false) values. The spatial reference system for `geom1` is used for spatial calculations.

9.3. Simple Features Relation Family (relation_family=Simple Features)

This clause establishes requirements for the *Simple Features* relation family.

Req 35 Implementations shall support `geof:sfEquals`, `geof:sfDisjoint`, `geof:sfIntersects`, `geof:sfTouches`, `geof:sfCrosses`, `geof:sfWithin`, `geof:sfContains`, `geof:sfOverlaps` as SPARQL extension functions, consistent with their corresponding DE-9IM intersection patterns, as defined by Simple Features [ISO19125-1].
`/req/geometry-topology-extension/sf-query-functions`

Boolean query functions defined for the Simple Features relation family, along with their associated DE-9IM intersection patterns, are shown in Table 6 below. Multi-row intersection patterns should be interpreted as a logical OR of each row. Each function accepts two arguments (`geom1` and `geom2`) of the geometry literal *serialization* type *specified* by serialization and version. Each function returns an `xsd:boolean` value of `true` if the specified relation exists between `geom1` and `geom2` and returns false otherwise. In each case, the spatial reference system of `geom1` is used for spatial calculations.

Table 6. Simple Features Query Functions

Query Function	Defining DE-9IM Intersection Pattern
`geof:sfEquals(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(TFFFTFFFT)`
`geof:sfDisjoint(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(FF*FF****)`
`geof:sfIntersects(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(FT******* F**T***** F***T****)`
`geof:sfTouches(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(FT******* F**T***** F***T****)`
`geof:sfCrosses(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(T*T***T**) for P/L, P/A, L/A; (0*T***T**) for L/L`
`geof:sfWithin(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(T*F**F***)`
`geof:sfContains(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(T*****FF*)`
`geof:sfOverlaps(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(T*T***T**) for A/A, P/P; (1*T***T**) for L/L`

9.4. Egenhofer Relation Family (relation_family=Egenhofer)

This clause establishes requirements for the *Egenhofer* relation family. Consult references [FORMAL] and [CATEG] for a more detailed discussion of *Egenhofer* relations.

Req 36 Implementations shall support `geof:ehEquals`, `geof:ehDisjoint`, `geof:ehMeet`, `geof:ehOverlap`, `geof:ehCovers`, `geof:ehCoveredBy`, `geof:ehInside`, `geof:ehContains` as SPARQL extension functions, consistent with their corresponding DE-9IM intersection patterns, as defined by Simple Features [ISO19125-1].
`/req/geometry-topology-extension/eh-query-functions`

Boolean query functions defined for the *Egenhofer* relation family, along with their associated DE-9IM intersection patterns, are shown in Table 7 below. Multi-row intersection patterns should be interpreted as a logical OR of each row. Each function accepts two arguments (`geom1` and `geom2`) of the geometry literal serialization type specified by *serialization* and *version*. Each function returns an `xsd:boolean` value of `true` if the specified relation exists between `geom1` and `geom2` and returns `false` otherwise. In each case, the spatial reference system of geom1 is used for spatial calculations.

Table 7. Egenhofer Query Functions

Query Function	Defining DE-9IM Intersection Pattern
`geof:ehEquals(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(TFFFTFFFT)`
`geof:ehDisjoint(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(FF*FF****)`
`geof:ehMeet(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(FT******* F**T***** F***T****)`
`geof:ehOverlap(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(T*T***T**)`
`geof:ehCovers(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(T*TFT*FF*)`
`geof:ehCoveredBy(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(TFF*TFT**)`
`geof:ehInside(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(TFF*FFT**)`
`geof:ehContains(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(T*TFF*FF*)`

9.5. Requirements for RCC8 Relation Family (relation_family=RCC8)

This clause establishes requirements for the *RCC8* relation family. Consult references [QUAL] and [LOGIC] for a more detailed discussion of *RCC8* relations.

Req 37 Implementations shall support `geof:rcc8eq`, `geof:rcc8dc`, `geof:rcc8ec`, `geof:rcc8po`, `geof:rcc8tppi`, `geof:rcc8tpp`, `geof:rcc8ntpp`, `geof:rcc8ntppi` as SPARQL extension functions, consistent with their corresponding DE-9IM intersection patterns, as defined by Simple Features [ISO19125-1].
`/req/geometry-topology-extension/rcc8-query-functions`

Boolean query functions defined for the *RCC8* relation family, along with their associated DE-9IM intersection patterns, are shown in Table 8 below. Each function accepts two arguments (`geom1` and `geom2`) of the geometry literal serialization type specified by *serialization* and *version*. Each function returns an `xsd:boolean` value of `true` if the specified relation exists between `geom1` and `geom2` and returns `false` otherwise. In each case, the spatial reference system of geom1 is used for spatial calculations.

Table 8. RCC8 Query Functions

Query Function	Defining DE-9IM Intersection Pattern
`geof:rcc8eq(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(TFFFTFFFT)`
`geof:rcc8dc(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(FFTFFTTTT)`
`geof:rcc8ec(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(FFTFTTTTT)`
`geof:rcc8po(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(TTTTTTTTT)`
`geof:rcc8tppi(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(TTTFTTFFT)`
`geof:rcc8tpp(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(TFFTTFTTT)`
`geof:rcc8ntpp(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(TFFTFFTTT)`
`geof:rcc8ntppi(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean`	`(TTTFFTFFT)`

10. RDFS Entailment Extension (relation_family, serialization, version)

This clause establishes the *RDFS Entailment Extension (relation_family, serialization, version)* parameterized requirements class, with IRI `/req/rdfs-entailment-extension`, which defines a mechanism for matching implicitly-derived RDF triples in GeoSPARQL queries. This class is parameterized to give implementations flexibility in the topological relation families and geometry types that they choose to support. This requirements class has a single corresponding conformance class *RDFS Entailment Extension (relation_family, serialization, version)*, `with IRI /conf/rdfs-entailment-extension`.

10.1. Parameters

- **relation_family**: Specifies the set of topological spatial relations to support.
- **serialization**: Specifies the serialization standard to use for geometry literals.
- **version**: Specifies the version of the serialization format used.

10.2. Common Requirements

The basic mechanism for supporting RDFS entailment has been defined by the W3C SPARQL 1.1 RDFS Entailment Regime [SPARQLENT].

Req 38 Basic graph pattern matching shall use the semantics defined by the RDFS Entailment Regime [SPARQLENT].
`/req/rdfs-entailment-extension/bgp-rdfs-ent`

10.3. WKT Serialization (serialization=WKT)

This section establishes the requirements for representing geometry data in RDF based on WKT as defined by Simple Features [ISO19125-1].

10.3.1. Geometry Class Hierarchy

The Simple Features specification presents a geometry class hierarchy. It is straightforward to represent this class hierarchy in RDFS and OWL by constructing IRIs for geometry classes using the following pattern: `http://www.opengis.net/ont/sf#{geometry class}` and by asserting appropriate `rdfs:subClassOf` statements.

The example RDF snippet below encodes the Polygon class from Simple Features 1.0.

```
sf:Polygon a rdfs:Class, owl:Class ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "Polygon"@en ;
    rdfs:subClassOf sf:Surface ;
    skos:definition "A planar surface defined by 1 exterior boundary and 0 or
                     more interior boundaries"@en .
```

Req 39 Implementations shall support graph patterns involving terms from an RDFS/OWL class hierarchy of geometry types consistent with the one in the specified version of Simple Features [ISO19125-1].

`/req/rdfs-entailment-extension/wkt-geometry-types`

10.4. GML Serialization (serialization=GML)

This section establishes requirements for representing geometry data in RDF based on GML as defined by Geography Markup Language Encoding Standard [OGC07-036].

10.4.1. Geometry Class Hierarchy

An RDF/OWL class hierarchy can be generated from the GML schema that implements `GM_Object` by constructing IRIs for geometry classes using the following pattern: `http://www.opengis.net/ont/gml#{GML Element}` and by asserting appropriate `rdfs:subClassOf` statements.

The example RDF snippet below encodes the Polygon class from GML 3.2.

```
gml:Polygon a rdfs:Class, owl:Class ;
    rdfs:isDefinedBy geo: ;
    skos:prefLabel "Polygon"@en ;
    rdfs:subClassOf gml:SurfacePatch ;
    skos:definition "A planar surface defined by 1 exterior boundary and 0 or
                     more interior boundaries."@en .
```

Req 40 Implementations shall support graph patterns involving terms from an RDFS/OWL class hierarchy of geometry types consistent with the GML schema that implements `GM_Object` using the specified *version* of GML [OGC07-036].

`/req/rdfs-entailment-extension/gml-geometry-types`

11. Query Rewrite Extension (relation_family, serialization, version)

This clause establishes the *Query Rewrite Extension (relation_family, serialization, version)* parameterized requirements class, with IRI `/req/query-rewrite-extension`, which has a single corresponding conformance class *Query Rewrite Extension (relation_family, serialization, version)*, with IRI `/conf/query-rewrite-extension`. This requirements class defines a set of RIF rules [RIF] that use topological extension functions defined in Clause 9 to establish the existence of direct topological predicates defined in Clause 7. One possible implementation strategy is to transform a given query by expanding a triple pattern involving a direct spatial predicate into a series of triple patterns and an invocation of the corresponding extension function as specified in the RIF rule.

The following rule specified using the RIF Core Dialect [RIFCORE] is used as a template to describe rules in the remainder of this clause. `ogc:relation` is used as a placeholder for the spatial relation IRIs defined in Clause 7, and `ogc:function` is used as a placeholder for the spatial functions defined in Clause 9.

```
Forall ?f1 ?f2 ?g1 ?g2 ?g1Serial ?g2Serial
    (?f1[ogc:relation->?f2] :-
        Or(
            And
                # feature  feature rule
                (?f1[geo:hasDefaultGeometry->?g1]
                 ?f2[geo:hasDefaultGeometry->?g2]
                 ?g1[ogc:asGeomLiteral->?g1Serial]
                 ?g2[ogc:asGeomLiteral->?g2Serial]
                 External(ogc:function (?g1Serial,?g2Serial)))
            And
                # feature  geometry rule
                (?f1[geo:hasDefaultGeometry->?g1]
                 ?g1[ogc:asGeomLiteral->?g1Serial]
                 ?f2[ogc:asGeomLiteral->?g2Serial]
                 External(ogc:function (?g1Serial,?g2Serial)))
            And
                # geometry - feature rule
                (?f2[geo:hasDefaultGeometry->?g2]
                 ?f1[ogc:asGeomLiteral->?g1Serial]
                 ?g2[ogc:asGeomLiteral->?g2Serial]
                 External(ogc:function (?g1Serial,?g2Serial)))
            And
                # geometry - geometry rule
                (?f1[ogc:asGeomLiteral->?g1Serial]
                 ?f2[ogc:asGeomLiteral->?g2Serial]
                 External(ogc:function (?g1Serial,?g2Serial)))
    )
)
```

NOTE	The GeoSPARQL 1.1 Standard contains a RIF rules artefact expanded for all function generated from this template and Python software for re-issuing the expanded artefact. See GeoSPARQL Standard structure.

11.1. Parameters

- **relation_family**: Specifies the set of topological spatial relations to support.
- **serialization**: Specifies the serialization standard to use for geometry literals.
- **version**: Specifies the version of the serialization format used.

11.2. Simple Features Relation Family (relation_family=Simple Features)

This clause defines requirements for the *Simple Features* relation family. Table 9 specifies the function and property substitutions for each rule in the *Simple Features* relation family.

Req 41 Basic graph pattern matching shall use the semantics defined by the RIF Core Entailment Regime [SPARQLENT] for the RIF rules [RIFCORE] `geor:sfEquals`, `geor:sfDisjoint`, `geor:sfIntersects`, `geor:sfTouches`, `geor:sfCrosses`, `geor:sfWithin`, `geor:sfContains`, `geor:sfOverlaps`.
`/req/query-rewrite-extension/sf-query-rewrite`

Table 9. Simple Features Query Transformation Rules

Rule	ogc:relation	ogc:function
geor:sfEquals	geo:sfEquals	geof:sfEquals
geor:sfDisjoint	geo:sfDisjoint	geof:sfDisjoint
geor:sfIntersects	geo:sfIntersects	geof:sfIntersects
geor:sfTouches	geo:sfTouches	geof:sfTouches
geor:sfCrosses	geo:sfCrosses	geof:sfCrosses
geor:sfWithin	geo:sfWithin	geof:sfWithin
geor:sfContains	geo:sfContains	geof:sfContains
geor:sfOverlaps	geo:sfOverlaps	geof:sfOverlaps

11.3. Egenhofer Relation Family (relation_family=Egenhofer)

This clause defines requirements for the *Egenhofer* relation family. Table 10 specifies the function and property substitutions for each rule in the *Egenhofer* relation family.

Req 42 Basic graph pattern matching shall use the semantics defined by the RIF Core Entailment Regime [SPARQLENT] for the RIF rules [RIFCORE] `geor:ehEquals`, `geor:ehDisjoint`, `geor:ehMeet`, `geor:ehOverlap`, `geor:ehCovers`, `geor:ehCoveredBy`, `geor:ehInside`, `geor:ehContains`.
`/req/query-rewrite-extension/eh-query-rewrite`

Table 10. Egenhofer Query Transformation Rules

Rule	ogc:relation	ogc:function
geor:ehEquals	geo:ehEquals	geof:ehEquals
geor:ehDisjoint	geo:ehDisjoint	geof:ehDisjoint
geor:ehMeet	geo:ehMeet	geof:ehMeet
geor:ehOverlap	geo:ehOverlap	geof:ehOverlap
geor:ehCovers	geo:ehCovers	geof:ehCovers
geor:ehCoveredBy	geo:ehCoveredBy	geof:ehCoveredBy
geor:ehInside	geo:ehInside	geof:ehInside
geor:ehContains	geo:ehContains	geof:ehContains

11.4. RCC8 Relation Family (relation_family=RCC8)

This clause defines requirements for the *RCC8* relation family. Table 11 specifies the function and property substitutions for each rule in the *RCC8* relation family.

Req 43 Basic graph pattern matching shall use the semantics defined by the RIF Core Entailment Regime [SPARQLENT] for the RIF rules [RIFCORE] `geor:rcc8eq`, `geor:rcc8dc`, `geor:rcc8ec`, `geor:rcc8po`, `geor:rcc8tppi`, `geor:rcc8tpp`, `geor:rcc8ntpp`, `geor:rcc8ntppi`.
`/req/query-rewrite-extension/rcc8-query-rewrite`

Table 11. RCC8 Query Transformation Rules

Rule	ogc:relation	ogc:function
geor:rcc8eq	geo:rcc8eq	geof:rcc8eq
geor:rcc8dc	geo:rcc8dc	geof:rcc8dc
geor:rcc8ec	geo:rcc8ec	geof:rcc8ec
geor:rcc8po	geo:rcc8po	geof:rcc8po
geor:rcc8tppi	geo:rcc8tppi	geof:rcc8tppi
geor:rcc8tpp	geo:rcc8tpp	geof:rcc8tpp
geor:rcc8ntpp	geo:rcc8ntpp	geof:rcc8ntpp
geor:rcc8ntppi	geo:rcc8ntppi	geof:rcc8ntppi

11.5. Special Considerations

The applicability of GeoSPARQL rules in certain circumstances has intentionally been left undefined.

The first situation arises for triple patterns with unbound predicates. Consider the query pattern below:

```
{ my:feature1 ?p my:feature2 }
```

When using a query transformation strategy, this triple pattern could invoke none of the GeoSPARQL rules or all of the rules. Implementations are free to support either of these alternatives.

The second situation arises when supporting GeoSPARQL rules in the presence of RDFS Entailment. The existence of a topological relation (possibly derived from a GeoSPARQL rule) can entail other RDF triples. For example, if `geo:sfOverlaps` has been defined as an `rdfs:subPropertyOf` the property `my:overlaps`, and the RDF triple `my:feature1 geo:sfOverlaps my:feature2` has been derived from a GeoSPARQL rule, then the RDF triple `my:feature1 my:overlaps my:feature2` can be entailed. Implementations may support such entailments but are not required to.

12. Future Work

Many future extensions of this standard are possible and, since the release of GeoSPARQL 1.0, many extensions have been made. The GeoSPARQL 1.1 release tried to incorporate many additions requested of the GeoSPARQL 1.0 Standard, including options the use of other serializations of geometry data (e.g. KML, GeoJSON, DGGS) and the addition of handling spatial scalar measurements.

Significant work remains in developing vocabularies for spatial data, and expanding the GeoSPARQL vocabularies with OWL axioms to aid in logical spatial reasoning would be a valuable contribution. There are also large amounts of existing feature data represented either in a GML file (or similar serialization) or in a datastore supporting the general feature model. It would be beneficial to develop standard processes for converting (or virtually converting and exposing) this data as RDF.

Annex A (normative) Abstract Test Suite

A.1 Conformance Class: *Core*

Conformance Class IRI: `/conf/core`

A.1.1 `/conf/core/sparql-protocol`

Requirement: `/req/core/sparql-protocol`

Implementations shall support the SPARQL Query Language for RDF [SPARQL], the SPARQL Protocol for RDF [SPARQLPROT] and the SPARQL Query Results XML Format [SPARQLRESX].

a. **Test purpose**: check conformance with this requirement
b. **Test method**: verify that the implementation accepts SPARQL queries and returns the correct results in the correct format, according to the SPARQL Query Language for RDF, the SPARQL Protocol for RDF and SPARQL Query Results XML Format W3C specifications.
c. **Reference**: Section 6.1
d. **Test Type**: Capabilities

A.1.2 `/conf/core/spatial-object-class`

Requirement: `/req/core/spatial-object-class`

Implementations shall allow the RDFS class geo:SpatialObject to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement
b. **Test method**: verify that queries involving geo:SpatialObject return the correct result on a test dataset.
c. **Reference**: Section 6.2.1
d. **Test Type**: Capabilities

A.1.3 `/conf/core/spatial-measure-class`

Requirement: `/req/core/spatial-measure-class` Implementations shall allow the RDFS class geo:SpatialMeasure to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement
b. **Test method**: verify that queries involving geo:SpatialMeasure return the correct result on a test dataset.
c. **Reference**: Section 6.2.3
d. **Test Type**: Capabilities

A.1.3 `/conf/core/feature-class`

Requirement: `/req/core/feature-class` Implementations shall allow the RDFS class geo:Feature to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving geo:Feature return the correct result on a test dataset.

c. **Reference**: Section 6.2.2

d. **Test Type**: Capabilities

A.2 Conformance Class: Topology Vocabulary Extension (relation_family)

Conformance Class IRI: `/conf/topology-vocab-extension`

A.2.1 relation_family = Simple Features

A.2.1.1 `/conf/topology-vocab-extension/sf-spatial-relations`

Requirement: `/req/topology-vocab-extension/sf-spatial-relations` Implementations shall allow the properties `geo:sfEquals`, `geo:sfDisjoint`, `geo:sfIntersects`, `geo:sfTouches`, `geo:sfCrosses`, `geo:sfWithin`, `geo:sfContains`, `geo:sfOverlaps` to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Verify that queries involving these properties return the correct result for a test dataset.
c. **Reference**: Section 7.2
d. **Test Type**: Capabilities

A.2.2 relation_family = Egenhofer

A.2.2.1 `/conf/topology-vocab-extension/eh-spatial-relations`

Requirement: `/req/topology-vocab-extension/eh-spatial-relations` Implementations shall allow the properties `geo:ehEquals`, `geo:ehDisjoint`, `geo:ehMeet`, `geo:ehOverlap`, `geo:ehCovers`, `geo:ehCoveredBy`, `geo:ehInside`, `geo:ehContains` to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Verify that queries involving these properties return the correct result for a test dataset.
c. **Reference**: Section 7.3
d. **Test Type**: Capabilities

A.2.3 relation_family = RCC8

A.2.3.1 `/conf/topology-vocab-extension/rcc8-spatial-relations`

Requirement: `/req/topology-vocab-extension/rcc8-spatial-relations` Implementations shall allow the properties `geo:rcc8eq`, `geo:rcc8dc`, `geo:rcc8ec`, `geo:rcc8po`, `geo:rcc8tppi`, `geo:rcc8tpp`, `geo:rcc8ntpp`, `geo:rcc8ntppi` to be used in SPARQL graph patterns

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Verify that queries involving these properties return the correct result for a test

dataset.

c. **Reference**: Section 7.4

d. **Test Type**: Capabilities

A.3 Conformance Class: Geometry Extension (serialization, version)

Conformance Class IRI: `/conf/geometry-extension`

A.3.1 Tests for all Serializations

A.3.1.1 `/conf/geometry-extension/geometry-class`

Requirement: `/req/geometry-extension/geometry-class` Implementations shall allow the RDFS class `geo:Geometry` to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement
b. **Test method**: verify that queries involving `geo:Geometry` return the correct result on a test dataset
c. **Reference**: Section 8.2.1
d. **Test Type**: Capabilities

A.3.1.2 `/conf/geometry-extension/feature-properties`

Requirement: `/req/geometry-extension/feature-properties` Implementations shall allow the properties `geo:hasGeometry` and `geo:hasDefaultGeometry`, `geo:hasLength`, `geo:hasArea`, `geo:hasVolume` `geo:hasBoundingBox` `geo:hasCentroid` to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Verify that queries involving these properties return the correct result for a test dataset.
c. **Reference**: Section 6.4
d. **Test Type**: Capabilities

A.3.1.3 `/conf/geometry-extension/geometry-properties`

Requirement: `/req/geometry-extension/geometry-properties` Implementations shall allow the properties `geo:dimension`, `geo:coordinateDimension`, `geo:spatialDimension`, `geo:isEmpty`, `geo:isSimple`, `geo:hasSerialization` to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Verify that queries involving these properties return the correct result for a test dataset.
c. **Reference**: Section 8.3
d. **Test Type**: Capabilities

A.3.1.4 `/conf/geometry-extension/query-functions`

Requirement: `/req/geometry-extension/query-functions` Implementations shall support `geof:distance`, `geof:buffer`, `geof:convexHull`, `geof:intersection`, `geof:union`, `geof:difference`, `geof:symDifference`, `geof:envelope` and `geof:boundary` as SPARQL extension functions, consistent with the definitions of the corresponding functions (distance, buffer, convexHull, intersection, difference, symDifference, envelope and boundary respectively) in Simple Features [ISO19125-1].

a. **Test purpose**: check conformance with this requirement

b. **Test method**: Verify that a set of SPARQL queries involving each of the following functions returns the correct result for a test dataset when using the specified serialization and version: `geof:distance`, `geof:buffer`, `geof:convexHull`, `geof:intersection`, `geof:union`, `geof:difference`, `geof:symDifference`, `geof:envelope` and `geof:boundary`.

c. **Reference**: Section 8.5

d. **Test Type**: Capabilities

A.3.1.5 `/conf/geometry-extension/srid-function`

Requirement: `/req/geometry-extension/srid-function` Implementations shall support `geof:getSRID` as a SPARQL extension function.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: Verify that a SPARQL query involving the `geof:getSRID` function returns the correct result for a test dataset when using the specified serialization and version.

c. **Reference**: Section 8.5.10

d. **Test Type**: Capabilities

A.3.2 serialization = WKT

A.3.2.1 `/conf/geometry-extension/wkt-literal`

Requirement: `/req/geometry-extension/wkt-literal` All `geo:wktLiteral` instances shall consist of an optional IRI identifying the Spatial Reference System (SRS) followed by Simple Features Well Known Text (WKT) describing a geometric value. Valid `geo:wktLiteral` instances are formed by concatenating a valid, absolute IRI as defined in [IETF3987], one or more spaces (Unicode U+0020 character) as a separator, and a WKT string as defined in Simple Features [ISO19125-1].

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving `geo:wktLiteral` values return the correct result for a test dataset.

c. **Reference**: Section 8.4.1.1

d. **Test Type**: Capabilities

A.3.2.2 `/conf/geometry-extension/wkt-literal-default-srs`

Requirement: `/req/geometry-extension/wkt-literal-default-srs` The IRI <http://www.opengis.net/def/crs/OGC/1.3/CRS84> shall be assumed as the SRS for `geo:wktLiterals` that do not specify an explicit SRS IRI.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving `geo:wktLiteral` values without an explicit encoded SRS IRI return the correct result for a test dataset.

c. **Reference**: Section 8.4.1.1

d. **Test Type**: Capabilities

A.3.2.3 `/conf/geometry-extension/wkt-axis-order`

Requirement: `/req/geometry-extension/wkt-axis-order` Coordinate tuples within `geo:wktLiterals` shall be interpreted using the axis order defined in the SRS used.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving `geo:wktLiteral` values return the correct result for a test dataset.

c. **Reference**: Section 8.4.1.1

d. **Test Type**: Capabilities

A.3.2.4 `/conf/geometry-extension/wkt-literal-empty`

Requirement: `/req/geometry-extension/wkt-literal-empty` An empty RDFS Literal of type `geo:wktLiteral` shall be interpreted as an empty geometry.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving empty `geo:wktLiteral` values return the correct result for a test dataset.

c. **Reference**: Section 8.4.1.1

d. **Test Type**: Capabilities

A.3.2.5 `/conf/geometry-extension/geometry-as-wkt-literal`

Requirement: `/req/geometry-extension/geometry-as-wkt-literal` Implementations shall allow the RDF property `geo:asWKT` to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving the `geo:asWKT` property return the correct result for a test dataset.

c. **Reference**: Section 8.4.1.2

d. **Test Type**: Capabilities

A.3.2.6 `/req/geometry-extension/asWKT-function`

Requirement: `/req/geometry-extension/asWKT-function` Implementations shall support `geof:asWKT`, as a SPARQL extension function

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Verify that a set of SPARQL queries involving the `geof:asWKT` function returns the correct result for a test dataset when using the specified serialization and version.
c. **Reference**: [_function_aswkt]
d. **Test Type**: Capabilities

A.3.3 serialization = GML

A.3.3.1 `/conf/geometry-extension/gml-literal`

Requirement: `/req/geometry-extension/gml-literal` All `geo:gmlLiteral` instances shall consist of a valid element from the GML schema that implements a subtype of GM_Object as defined in [OGC 07-036].

a. **Test purpose**: check conformance with this requirement
b. **Test method**: verify that queries involving `geo:gmlLiteral` values return the correct result for a test dataset.
c. **Reference**: Section 8.4.2.1
d. **Test Type**: Capabilities

A.3.3.2 `/conf/geometry-extension/gml-literal-empty`

Requirement: `/req/geometry-extension/gml-literal-empty` An empty `geo:gmlLiteral` shall be interpreted as an empty geometry.

a. **Test purpose**: check conformance with this requirement
b. **Test method**: verify that queries involving empty `geo:gmlLiteral` values return the correct result for a test dataset.
c. **Reference**: Section 8.4.2.1
d. **Test Type**: Capabilities

A.3.3.3 `/conf/geometry-extension/gml-profile`

Requirement: `/req/geometry-extension/gml-profile` Implementations shall document supported GML profiles.

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Examine the implementation's documentation to verify that the supported GML profiles are documented.

c. **Reference**: Section 8.4.2.1

d. **Test Type**: Documentation

A.3.3.4 `/conf/geometry-extension/geometry-as-gml-literal`

Requirement: `/req/geometry-extension/geometry-as-gml-literal` Implementations shall allow the RDF property `geo:asGML` to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving the `geo:asGML` property return the correct result for a test dataset.

c. **Reference**: Section 8.4.2.2

d. **Test Type**: Capabilities

A.3.3.5 `/req/geometry-extension/asGML-function`

Requirement: `/req/geometry-extension/asGML-function` Implementations shall support `geof:asGML`, as a SPARQL extension function

a. **Test purpose**: check conformance with this requirement

b. **Test method**: Verify that a set of SPARQL queries involving the `geof:asGML` function returns the correct result for a test dataset when using the specified serialization and version.

c. **Reference**: [_function_asgml]

d. **Test Type**: Capabilities

A.3.4 serialization = GEOJSON

A.3.4.1 `/req/geometry-extension/geojson-literal`

Requirement: `/req/geometry-extension/geojson-literal` All `geo:geoJSONLiteral` instances shall consist of valid JSON that conforms to the GeoJSON specification [GEOJSON]

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving `geo:geoJSONLiteral` values return the correct result for a test dataset.

c. **Reference**: Section 8.4.2.2

d. **Test Type**: Capabilities

A.3.4.2 `/req/geometry-extension/geojson-literal-srs`

Requirement: `/req/geometry-extension/geojson-literal-default-srs` The IRI <http://www.opengis.net/def/crs/OGC/1.3/CRS84> shall be assumed as the SRS for `geo:geoJSONLiteral` instances that do not specify an explicit SRS IRI.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving `geo:geoJSONLiteral` values without an explicit encoded SRS IRI return the correct result for a test dataset.

c. **Reference**: Section 8.4.3.1

d. **Test Type**: Capabilities

A.3.4.3 `/req/geometry-extension/geojson-literal-empty`

Requirement: `/req/geometry-extension/geojson-literal-empty` An empty `geo:geoJSONLiteral` shall be interpreted as an empty geometry.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving empty `geo:geoJSONLiteral` values return the correct result for a test dataset.

c. **Reference**: Section 8.4.3.1

d. **Test Type**: Capabilities

A.3.4.4 `/req/geometry-extension/geometry-as-geojson-literal`

Requirement: `/req/geometry-extension/geometry-as-geojson-literal` Implementations shall allow the RDF property `geo:asGeoJSON` to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving the `geo:asGeoJSON` property return the correct result for a test dataset.

c. **Reference**: Section 8.4.3.2

d. **Test Type**: Capabilities

A.3.4.5 `/req/geometry-extension/asGeoJSON-function`

Requirement: `/req/geometry-extension/asGeoJSON-function` Implementations shall support `geof:asGeoJSON`, as a SPARQL extension function

a. **Test purpose**: check conformance with this requirement

b. **Test method**: Verify that a set of SPARQL queries involving the `geof:asGeoJSON` function returns the correct result for a test dataset when using the specified serialization and version.

c. **Reference**: [_function_asgeojson]

d. **Test Type**: Capabilities

A.3.5 serialization = KML

A.3.5.1 `/req/geometry-extension/kml-literal`

Requirement: `/req/geometry-extension/kml-literal` All `geo:kmlLiteral` instances shall consist of a valid element from the KML schema that implements a `kml:AbstractObjectGroup` as defined in

[OGCKML].

a. **Test purpose**: check conformance with this requirement
b. **Test method**: verify that queries involving `geo:kmlLiteral` values return the correct result for a test dataset.
c. **Reference**: [_rdfs_datatype_geomklliteral]
d. **Test Type**: Capabilities

A.3.5.2 `/req/geometry-extension/kml-literal-srs`

Requirement: `/req/geometry-extension/kml-literal-default-srs` The IRI <http://www.opengis.net/def/crs/OGC/1.3/CRS84> shall be assumed as the SRS for `geo:kmlLiterals` that do not specify an explicit SRS IRI.

a. **Test purpose**: check conformance with this requirement
b. **Test method**: verify that queries involving `geo:kmlLiteral` values without an explicit encoded SRS IRI return the correct result for a test dataset.
c. **Reference**: [_rdfs_datatype_geomklliteral]
d. **Test Type**: Capabilities

A.3.5.3 `/req/geometry-extension/kml-literal-empty`

Requirement: `/req/geometry-extension/kml-literal-empty` An empty `geo:kmlLiteral` shall be interpreted as an empty geometry.

a. **Test purpose**: check conformance with this requirement
b. **Test method**: verify that queries involving empty `geo:kmlLiteral` values return the correct result for a test dataset.
c. **Reference**: [_rdfs_datatype_geomklliteral]
d. **Test Type**: Capabilities

A.3.5.4 `/req/geometry-extension/geometry-as-kml-literal`

Requirement: `/req/geometry-extension/geometry-as-kml-literal` Implementations shall allow the RDF property `geo:asKML` to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement
b. **Test method**: verify that queries involving the `geo:asKML` property return the correct result for a test dataset.
c. **Reference**: Section 8.4.4.2
d. **Test Type**: Capabilities

A.3.5.5 `/req/geometry-extension/asKML-function`

Requirement: `/req/geometry-extension/asKML-function` Implementations shall support `geof:asKML`, as a SPARQL extension function

a. **Test purpose**: check conformance with this requirement

b. **Test method**: Verify that a set of SPARQL queries involving the `geof:asKML` function returns the correct result for a test dataset when using the specified serialization and version.

c. **Reference**: [_function_askml]

d. **Test Type**: Capabilities

A.3.6 serialization = DGGS

A.3.6.1 `/req/geometry-extension/dggswkt-literal`

Requirement: `/req/geometry-extension/dggswkt-literal` All RDFS Literals of type `geo:dggsLiteral` shall consist of a DGGS identifier, an IRI enclosed in '<' & '>', followed by one or more spaces (Unicode U+0020 character) and a DGGS geometry serialization formulated according to the identified DGGS.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving `geo:dggsLiteral` values return the correct result for a test dataset.

c. **Reference**: [_dggs_serialization_serializationdggs]

d. **Test Type**: Capabilities

A.3.6.2 `/req/geometry-extension/dggswkt-literal-empty`

Requirement: `/req/geometry-extension/dggswkt-literal-empty` An empty `geo:dggsLiteral` shall be interpreted as an empty geometry.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving empty `geo:dggsLiteral` values return the correct result for a test dataset.

c. **Reference**: [_dggs_serialization_serializationdggs]

d. **Test Type**: Capabilities

A.3.6.3 `/req/geometry-extension/geometry-as-dggswkt-literal`

Requirement: `/req/geometry-extension/geometry-as-dggswkt-literal` Implementations shall allow the RDF property `geo:asDGGSWKT` to be used in SPARQL graph patterns.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: verify that queries involving the `geo:asDGGSWKT` property return the correct result for a test dataset.

c. **Reference**: [_property_geoasdggswkt]

d. **Test Type**: Capabilities

A.3.6.4 `/req/geometry-extension/asDggsWKT-function`

Requirement: `/req/geometry-extension/asDggsWKT-function` Implementations shall support `geof:asDGGSWKT`, as a SPARQL extension function

a. **Test purpose**: check conformance with this requirement

b. **Test method**: Verify that a set of SPARQL queries involving the `geof:asDGGSWKT` function returns the correct result for a test dataset when using the specified serialization and version.

c. **Reference**: [_function_asdggswkt]

d. **Test Type**: Capabilities

/req/geometry-extension/asDggsWKT-function

A.4 Conformance Class: Geometry Topology Extension (relation_family, serialization, version)

Conformance Class IRI: `/conf/geometry-topology-extension`

A.4.1 Tests for all relation families

A.4.1.1 `/conf/geometry-topology-extension/relate-query-function`

Requirement: `/req/geometry-topology-extension/relate-query-function` Implementations shall support `geof:relate` as a SPARQL extension function, consistent with the relate operator defined in Simple Features [ISO19125-1].

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Verify that a set of SPARQL queries involving the `geof:relate` function returns the correct result for a test dataset when using the specified serialization and version.
c. **Reference**: Section 9.2
d. **Test Type**: Capabilities

A.4.2 relation_family = Simple Features

A.4.2.1 `/conf/geometry-topology-extension/sf-query-functions`

Requirement: `/req/geometry-topology-extension/sf-query-functions` Implementations shall support `geof:sfEquals`, `geof:sfDisjoint`, `geof:sfIntersects`, `geof:sfTouches`, `geof:sfCrosses`, `geof:sfWithin`, `geof:sfContains`, `geof:sfOverlaps` as SPARQL extension functions, consistent with their corresponding DE-9IM intersection patterns, as defined by Simple Features [ISO19125-1].

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Verify that a set of SPARQL queries involving each of the following functions returns the correct result for a test dataset when using the specified serialization and version: `geof:sfEquals`, `geof:sfDisjoint`, `geof:sfIntersects`, `geof:sfTouches`, `geof:sfCrosses`, `geof:sfWithin`, `geof:sfContains`, `geof:sfOverlaps` .
c. **Reference**: Section 7.2
d. **Test Type**: Capabilities

A.4.3 relation_family = Egenhofer

A.4.3.1 `/conf/geometry-topology-extension/eh-query-functions`

Requirement: `/req/geometry-topology-extension/eh-query-functions` Implementations shall

support `geof:ehEquals`, `geof:ehDisjoint`, `geof:ehMeet`, `geof:ehOverlap`, `geof:ehCovers`, `geof:ehCoveredBy`, `geof:ehInside`, `geof:ehContains` as SPARQL extension functions, consistent with their corresponding DE-9IM intersection patterns, as defined by Simple Features [ISO 19125- 1].

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Verify that a set of SPARQL queries involving each of the following functions returns the correct result for a test dataset when using the specified serialization and version: `geof:ehEquals`, `geof:ehDisjoint`, `geof:ehMeet`, `geof:ehOverlap`, `geof:ehCovers`, `geof:ehCoveredBy`, `geof:ehInside`, `geof:ehContains`.
c. **Reference**: Section 7.3
d. **Test Type**: Capabilities

A.4.4 relation_family = RCC8

A.4.4.1 `/conf/geometry-topology-extension/rcc8-query-functions`

Requirement: `/req/geometry-topology-extension/rcc8-query-functions` Implementations shall support `geof:rcc8eq`, `geof:rcc8dc`, `geof:rcc8ec`, `geof:rcc8po`, `geof:rcc8tppi`, `geof:rcc8tpp`, `geof:rcc8ntpp`, `geof:rcc8ntppi` as SPARQL extension functions, consistent with their corresponding DE-9IM intersection patterns, as defined by Simple Features [ISO19125-1].

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Verify that a set of SPARQL queries involving each of the following functions returns the correct result for a test dataset when using the specified serialization and version: `geof:rcc8eq`, `geof:rcc8dc`, `geof:rcc8ec`, `geof:rcc8po`, `geof:rcc8tppi`, `geof:rcc8tpp`, `geof:rcc8ntpp`, `geof:rcc8ntppi` .
c. **Reference**: Section 7.4
d. **Test Type**: Capabilities

A.5 Conformance Class: RDFS Entailment Extension (relation_family, serialization, version)

Conformance Class IRI: `/conf/rdfs-entailment-extension`

A.5.1 Tests for all implementations

A.5.1.1 `/conf/rdfsentailmentextension/bgp-rdfs-ent`

Requirement: `/req/rdfs-entailment-extension/bgp-rdfs-ent` Basic graph pattern matching shall use the semantics defined by the RDFS Entailment Regime [SPARQLENT].

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Verify that a set of SPARQL queries involving entailed RDF triples returns the correct result for a test dataset using the specified serialization, version and relation_family.
c. **Reference**: Section 10.2
d. **Test Type**: Capabilities

A.5.2 serialization=WKT

A.5.2.1 `/conf/rdfs-entailment-extension/wkt-geometry-types`

Requirement: `/req/rdfs-entailment-extension/wkt-geometry-types` Implementations shall support graph patterns involving terms from an RDFS/OWL class hierarchy of geometry types consistent with the one in the specified version of Simple Features [ISO19125-1].

a. **Test purpose**: check conformance with this requirement
b. **Test method**: Verify that a set of SPARQL queries involving WKT Geometry types returns the correct result for a test dataset using the specified version of Simple Features.
c. **Reference**: Section 10.3.1
d. **Test Type**: Capabilities

A.5.3 serialization=GML

A.5.3.1 `/conf/rdfs-entailment-extension/gml-geometry-types`

Requirement: `/req/rdfs-entailment-extension/gml-geometry-types` Implementations shall support graph patterns involving terms from an RDFS/OWL class hierarchy of geometry types consistent with the GML schema that implements GM_Object using the specified version of GML [OGC07-036].

a. **Test purpose**: check conformance with this requirement

b. **Test method**: Verify that a set of SPARQL queries involving GML Geometry types returns the correct result for a test dataset using the specified version of GML.

c. **Reference**: Section 10.4.1

d. **Test Type**: Capabilities

A.6 Conformance Class: Query Rewrite Extension (relation_family, serialization, version)

Conformance Class IRI: `/conf/query-rewrite-extension`

A.6.1 relation_family = Simple Features

A.6.1.1 `/conf/query-rewrite-extension/sf-query-rewrite`

Requirement: `/req/query-rewrite-extension/sf-query-rewrite` Basic graph pattern matching shall use the semantics defined by the RIF Core Entailment Regime [SPARQLENT] for the RIF rules [RIFCORE] `geor:sfEquals`, `geor:sfDisjoint`, `geor:sfIntersects`, `geor:sfTouches`, `geor:sfCrosses`, `geor:sfWithin`, `geor:sfContains` and `geor:sfOverlaps`..

a. **Test purpose**: check conformance with this requirement

b. **Test method**: Verify that queries involving the following query transformation rules return the correct result for a test dataset when using the specified serialization and version: `geor:sfEquals`, `geor:sfDisjoint`, `geor:sfIntersects`, `geor:sfTouches`, `geor:sfCrosses`, `geor:sfWithin`, `geor:sfContains` and `geor:sfOverlaps`.

c. **Reference**: Section 9.3

d. **Test Type**: Capabilities

A.6.2 relation_family = Egenhofer

A.6.2.1 `/conf/query-rewrite-extension/eh-query-rewrite`

Requirement: `/req/query-rewrite-extension/eh-query-rewrite` Basic graph pattern matching shall use the semantics defined by the RIF Core Entailment Regime [SPARQLENT] for the RIF rules [RIFCORE] `geor:ehEquals`, `geor:ehDisjoint`, `geor:ehMeet`, `geor:ehOverlap`, `geor:ehCovers`, `geor:ehCoveredBy`, `geor:ehInside`, `geor:ehContains`.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: Verify that queries involving the following query transformation rules return the correct result for a test dataset when using the specified serialization and version: `geor:ehEquals`, `geor:ehDisjoint`, `geor:ehMeet`, `geor:ehOverlap`, `geor:ehCovers`, `geor:ehCoveredBy`, `geor:ehInside`, `geor:ehContains`.

c. **Reference**: Section 9.4

d. **Test Type**: Capabilities

A.6.3 relation_family = RCC8

A.6.3.1 `/conf/query-rewrite-extension/rcc8-query-rewrite`

Requirement: `/req/query-rewrite-extension/rcc8-query-rewrite` Basic graph pattern matching shall use the semantics defined by the RIF Core Entailment Regime [SPARQLENT] for the RIF rules [RIFCORE] `geor:rcc8eq`, `geor:rcc8dc`, `geor:rcc8ec`, `geor:rcc8po`, `geor:rcc8tppi`, `geor:rcc8tpp`, `geor:rcc8ntpp`, `geor:rcc8ntppi`.

a. **Test purpose**: check conformance with this requirement

b. **Test method**: Verify that queries involving the following query transformation rules return the correct result for a test dataset when using the specified serialization and version: `geor:rcc8eq`, `geor:rcc8dc`, `geor:rcc8ec`, `geor:rcc8po`, `geor:rcc8tppi`, `geor:rcc8tpp`, `geor:rcc8ntpp`, `geor:rcc8ntppi`.

c. **Reference**: Section 11.4

d. **Test Type**: Capabilities

Annex B (informative) GeoSPARQL Examples

B.1 RDF Examples

This Section illustrates GeoSPARQL ontology modelling with extended examples.

New Features checklist - to be removed when all examples are complete:

New element	Section
Classes	
Spatial Measure class	Section 6.2.3

B.1.1 Classes

B 1.1.1 `SpatialObject`

The `SpatialObject` class is defined in Section 6.2.1.

Basic use (as per the example in the class definition)

```
eg:x a geo:SpatialObject ;
    skos:prefLabel "Object X";
.
```

NOTE	It is unlikely that users of GeoSPARQL will create many instances of `geo:SpatialObject` as its two more concrete subclasses, `geo:Feature` & `geo:Geometry`, are more directly relatable to real-world phenomena and use.

B 1.1.2 `Feature`

The `Feature` class is defined in Section 6.2.2.

B 1.1.2.1 Basic use

```
eg:x a geo:Feature ;
    skos:prefLabel "Feature X";
.
```

Here a `Feature` is declared and given a preferred label.

B 1.1.2.2 A `Feature` related to a `Geometry`

```
eg:x a geo:Feature ;
    skos:prefLabel "Feature X";
    geo:hasGeometry [
        geo:asWKT "MULTIPOLYGON (((149.06016596 -35.23610602, 149.060620714
-35.236043434, ... , 149.06016596 -35.23610602)))"^^geo:wktLiteral ;
    ] ;
.
```

Here a `Feature` is declared, given a preferred label and a geometry for that `Feature` is indicated with the use of `geo:hasGeometry`. The `Geometry` indicated is described using a *Well-Known Text* literal value, indicated by the property `geo:asWKT` and the literal type `geo:wktLiteral`.

B 1.1.2.3 `Feature` with both `Geometry` and `SpatialMeasure` instances indicated

```
@prefix qudt: <http://qudt.org/schema/qudt/> .

eg:x a geo:Feature ;
    skos:prefLabel "Feature X";
        geo:hasGeometry [
            geo:asWKT "MULTIPOLYGON (((149.06016596 -35.23610602, 149.060620714
-35.236043434, ... , 149.06016596 -35.23610602)))"^^geo:wktLiteral ;
    ] ;
    geo:hasArea [
        a geo:SpatialMeasure ;
        qudt:numericValue 8.7 ;
        qudt:unit <http://qudt.org/vocab/unit/HA> ;  # hectare
    ] ;
.
```

Here a `Feature` and a `Geometry` are described as per the previous example but an additional `SpatialMeasure` for the `Feature` is indicated with the use of the property `geo:hasArea`. This property indicates a `SpatialMeasure` instance which conveys its value using the *Quantities, Units, Dimensions and Types (QUDT)* ontology[14]

Note that in this example, the use of QUDT and its `qudt:numericValue` & `qudt:unit` is just one of many possible ways to convey a `SpatialMeasure` instance's value.

B 1.1.2.4 `Feature` with two different `Geometry` instances indicated

```
@prefix qudt: <http://qudt.org/schema/qudt/> .

eg:x a geo:Feature ;
    skos:prefLabel "Feature X";
    geo:hasGeometry [
        rdfs:label "Official boundary" ;
        rdfs:comment "Official boundary from the Department of Xxx" ;
        geo:asWKT "MULTIPOLYGON (((149.06016596 -35.23610602, 149.060620714
-35.236043434, ... , 149.06016596 -35.23610602)))"^^geo:wktLiteral ;
    ] ,
    [
        rdfs:label "Unofficial boundary" ;
        rdfs:comment "Unofficial boundary as actually used by everyone" ;
        geo:asWKT "MULTIPOLYGON (((149.06016597 -35.23610603, 149.060620715
-35.236043435, ... , 149.06016597 -35.23610603)))"^^geo:wktLiteral ;
    ] ;
.
```

In this example, `Feature X` has two different `Geometry` instances indicated with their different explained in annotation properties. No ontology properties are used to indicate a difference in these `Geometry` thus machine use of this `Feature` woud not be easily able to differentiate them.

B 1.1.2.5 `Feature` with two different `Geometry` instances with different property values

```
@prefix qudt: <http://qudt.org/schema/qudt/> .

eg:x a geo:Feature ;
    skos:prefLabel "Feature X";
    geo:hasGeometry [
        geo:hasSpatialResolution [
            qudt:numericValue 100 ;
            qudt:unit <http://qudt.org/vocab/unit/M> ;  # metre
        ] ;
        geo:asWKT "MULTIPOLYGON (((149.06016 -35.23610, 149.060620 -35.236043, ... ,
149.06016 -35.23610)))"^^geo:wktLiteral ;
    ] ,
    [
        geo:hasSpatialResolution [
            qudt:numericValue 5 ;
            qudt:unit <http://qudt.org/vocab/unit/M> ;  # metre
        ] ;
        geo:asWKT "MULTIPOLYGON (((149.06016597 -35.23610603, 149.060620715
-35.236043435, ... , 149.06016597 -35.23610603)))"^^geo:wktLiteral ;
    ] ;
.
```

In this example, `Feature X` has two different `Geometry` instances indicated with different spatial resolutions. This use of the `geo:hasSpatialResolution` property follows Example B 1.1.2.3 `Feature`

with both Geometry and SpatialMeasure instances indicated with its use of QUDT for unit & value.

Machine use of this Feature would be able to differentiate the two Geometry instnaces based on this use of geo:hasSpatialResolution.

B 1.1.2.6 Feature with two different types of Geometry instances

```
eg:x a geo:Feature ;
    skos:prefLabel "Feature X";
    geo:hasGeometry [
        geo:asWKT "POLYGON ((149.06016 -35.23610, 149.060620 -35.236043, ... ,
149.06016 -35.23610))"^^geo:wktLiteral ;
    ] ;
    geo:hasCentroid [
        geo:asWKT "POINT (149.06017784 -35.23612321)"^^geo:WktLiteral ;
    ] ;
.
```

Here a Feature instance has two geometries, one indicated with the general property hasGeometry and a second indicated with the specialised property hasCentroid which suggests the role that the indicated geometry plays. Note that while hasGeometry may indicate any type of Geometry, hasCentroid should only be used to indicate a point geometry. It may be informally inferred that the polygonal geometry is the Feature instance's boundary.

B.1.1.3 Geometry

The Geometry class is defined in Section 8.2.1.

B1.1.3.1 Basic Use

```
eg:y a geo:Geometry ;
    skos:prefLabel "Geometry Y";
.
```

Here a Geometry is declared and given a preferred label.

From GeoSPARQL 1.0 use, the most commonly observed use of a Geometry is in relation to a Feature as per the example in B 1.1.2.2 A Feature related to a Geometry and often the Geometry is indirectly declared by the use of hasGeometry on the Feature instance indicating a Blank Node, however it is entirely possible to declare Geometry instances without any Feature instances. The next basic example declares a Geometry instance with an demonstration absolute URI and data.

```
<https://example.com/geometry/y>
    a geo:Geometry ;
    skos:prefLabel "Geometry Y";
    geo:asWKT "MULTIPOLYGON (((149.06016 -35.23610, 149.060620 -35.236043, ... ,
149.06016 -35.23610)))"^^geo:wktLiteral ;
    geo:inSRS <http://www.opengis.net/def/crs/EPSG/0/4326> ;
.
```

Here the `Geometry` instance has data in WKT form and declares an SRS, WGS84: using the EPSG identifier.

B1.1.3.2 A `Geometry` with multiple serializations

```
eg:x
    a geo:Feature ;
    skos:prefLabel "Feature X";
    geo:hasGeometry [
        geo:asWKT "<http://www.opengis.net/def/crs/EPSG/0/4326> MULTIPOLYGON
(((149.06016 -35.23610, 149.060620 -35.236043, ... , 149.06016
-35.23610)))"^^geo:wktLiteral ;
        geo:asDGGS "<https://w3id.org/dggs/aspix> CELLLIST ((R1234 R1235 R1236 ...
R1256))"^^geo:dggsLiteral ;
    ] ;
.
```

Here a single `Geometry`, linked to a `Feature` instance is expressed using two different serializations: Well-known Text and the AusPIX DGGS. Note that `inSRS` is not used for the `Geometry` instance as this would conflict with the DGGS serialization so, as per GeoSPARQL 1.0, the SRS used for the WKT serialization is included directly in the WKT literal value.

B 1.1.4 `SpatialMeasure`

The `SpatialMeasure` class is defined in Section 6.2.3.

B1.1.4.1 Basic Use

```
@prefix qudt: <http://qudt.org/schema/qudt/> .

eg:z
    a geo:SpatialMeasure ;
    skos:prefLabel "Area Z" ;
    qudt:numericValue 8.7 ;
    qudt:unit <http://qudt.org/vocab/unit/HA> ;  # hectare
```

This example defines an instance of `SpatialMeasure` and supplies it with a preferred label, a numeric value and a unit of measure.

`SpatialMeasure` instances may be declared in isolation like this - without reference to a `Feature` instance, just as Geometry instances may be - and future use of GeoSPARQL may see such declarations used widely however, at the time of writing GeoSPARQL 1.1, anticipated use of `SpatialMeasure` is with reference to a `Feature` instance: the *thing* for which the spatial measure is defined. The next example give use in relation to a `Feature` instance.

```
@prefix qudt: <http://qudt.org/schema/qudt/> .

eg:x
    a geo:Feature ;
    skos:prefLabel "Feature X" ;
    geo:hasArea [
        a geo:SpatialMeasure ;
        qudt:numericValue 8.7 ;
        qudt:unit <http://qudt.org/vocab/unit/HA> ;  # hectare
    ] ;
```

In this example, the `SpatialMeasure` instance has no label - only a numeric value and a unit of measure - but it is declared to be the area for "Feature X".

B1.1.4.1 Multiple measures

```
@prefix qudt: <http://qudt.org/schema/qudt/> .

eg:x
    a geo:Feature ;
    skos:prefLabel "Lake X" ;
    eg:hasFeatureCategory <http://example.com/cat/lake> ;
    geo:hasArea [
        a geo:SpatialMeasure ;
        qudt:numericValue 8.7 ;
        qudt:unit <http://qudt.org/vocab/unit/HA> ;  # hectare
    ] ;
    geo:hasVolume [
        a geo:SpatialMeasure ;
        qudt:numericValue 624432 ;
        qudt:unit <http://qudt.org/vocab/unit/M3> ;  # cubic metre
    ]
```

This example shows a `Feature` instance with area and volume declared. A categorization of the feature is given through the use of the `eg:hasFeatureCategory` dummy property which, along with the feature's preferred label, indicate that this feature is a lake. Having both an area and a volume makes sense for a lake.

B.1.2 Properties

New Features checklist - to be removed when all examples are complete:

New element	Section
Feature Properties	
hasBoundingBox	Section 6.4.3
hasCentroid	Section 6.4.4
hasLength	Section 6.4.5
hasArea	Section 6.4.6
hasVolume	Section 6.4.7
Geometry Properties	
inSRS	Section 8.3.8
Geometry Serializations	
geoJSONLiteral	Section 8.4.3.1
asGeoJSON	Section 8.4.3.2
kmlLiteral	Section 8.4.4.1
asKML	Section 8.4.4.2
dggsWKTLiteral	[RDFS Datatype: geo:dggsWKTLiteral]
asDGGS	Section 8.4.5.2

B.1.2.1 Feature Properties

This example shows a `Feature` instance with each of the properties defined in [8.3. Standard Properties for geo:Feature] used.

```
eg:x
    a geo:Feature ;
    skos:preferredLabel "Feature X" ;
    geo:hasGeometry [
        geo:asWKT "<http://www.opengis.net/def/crs/EPSG/0/4326> POLYGON ((149.06016
-35.23610, ... , 149.06016 -35.23610)))"^^geo:wktLiteral ;
    ] ;
    geo:hasDefaultGeometry [
        geo:asWKT "<http://www.opengis.net/def/crs/EPSG/0/4326> POLYGON ((149.0601
-35.2361, ... , 149.0601 -35.2361)))"^^geo:wktLiteral ;
    ] ;
    geo:hasLength [
        a geo:SpatialMeasure ;
        qudt:numericValue 355 ;
        qudt:unit <http://qudt.org/vocab/unit/M> ;  # metre
    ] ;
    geo:hasArea [
        a geo:SpatialMeasure ;
        qudt:numericValue 8.7 ;
        qudt:unit <http://qudt.org/vocab/unit/HA> ;  # hectare
    ] ;
    geo:hasVolume [
        a geo:SpatialMeasure ;
        qudt:numericValue 624432 ;
        qudt:unit <http://qudt.org/vocab/unit/M3> ;  # cubic metre
    ]
    geo:hasCentroid [
        geo:asWKT "POINT (149.06017784 -35.23612321)"^^geo:wktLiteral ;
    ] ;
    geo:hasBoundingBox [
        geo:asWKT "<http://www.opengis.net/def/crs/EPSG/0/4326> POLYGON ((149.060
-35.236, ... , 149.060 -35.236)))"^^geo:wktLiteral ;
    ] ;
    geo:hasSpatialResolution [
        qudt:numericValue 5 ;
        qudt:unit <http://qudt.org/vocab/unit/M> ;  # metre
    ] ;
.
```

The properties defined for this example's `Feature` instance are vaguely aligned in that the values are not real but are not unrealistic either. It is outside the scope of GeoSPARQL to validate `Feature` instances' property values.

B.1.2.2 Geometry Properties

This example shows a `Geometry` instance declaread in relation to a `Feature` instance with each of the properties defined in [8.4. Standard Properties for geo:Geometry] used.

```
eg:x
    a geo:Feature ;
    geo:hasGeometry [
        skos:prefLabel "Geometry Y" ;
        geo:dimension 2 ;
        geo:coordinateDimension 2 ;
        geo:spatialDimension 2 ;
        geo:isEmpty false ;
        geo:isSimple true ;
        geo:hasSerialization "<http://www.opengis.net/def/crs/EPSG/0/4326> POLYGON
((149.060 -35.236, ... , 149.060 -35.236)))"^^geo:wktLiteral ;
        geo:inSRS <http://www.opengis.net/def/crs/EPSG/0/4326> ;
    ] ;
.
```

In this example, each of the standards properties defined for a `Geometry` instance has realistic values, for example, the `geo:isEmpty` is set to `false` since the geometry contains information. Note that the SRS of the given serialzation is given within the geometry literal (a WKT value) as well as by the `geo:inSRS` property.

B.1.2.3 Geometry Serializations

This section shows a `Geometry` instance for a `Feature` instance which is represented in all supported GeoSPARQL serlializations. The geometry values given are real geometry values and approximate Moreton Island in Queensland, Australia.

```
eg:x
    a geo:Feature ;
    geo:hasGeometry [
        geo:asWKT """<http://www.opengis.net/def/crs/EPSG/0/4326>
            POLYGON ((
                153.3610112 -27.0621757,
                153.3658177 -27.1990606,
                153.421436 -27.3406573,
                153.4269292 -27.3607835,
                153.4434087 -27.3315078,
                153.4183848 -27.2913403,
                153.4189391 -27.2039578,
                153.4673476 -27.0267166,
                153.3610112 -27.0621757
            ))"""^^geo:wktLiteral ;

        geo:asGML """<gml:Polygon
                srsName="http://www.opengis.net/def/crs/EPSG/0/4326">
                <gml:exterior>
                    <gml:LinearRing>
                        <gml:posList>
                            -27.0621757 153.3610112
                            -27.1990606 153.3658177
```

```
                            -27.3406573 153.421436
                            -27.3607835 153.4269292
                            -27.3315078 153.4434087
                            -27.2913403 153.4183848
                            -27.2039578 153.4189391
                            -27.0267166 153.4673476
                            -27.0621757 153.3610112
                        </gml:posList>
                    </gml:LinearRing>
                </gml:exterior>
            </gml:Polygon>"""^^go:gmlLiteral ;

        geo:asKML """<Polygon>
                <outerBoundaryIs>
                    <LinearRing>
                        <coordinates>
                        153.3610112,-27.0621757
                        153.3658177,-27.1990606
                        153.421436,-27.3406573
                        153.4269292,-27.3607835
                        153.4434087,-27.3315078
                        153.4183848,-27.2913403
                        153.4189391,-27.2039578
                        153.4673476,-27.0267166
                        153.3610112,-27.0621757
                        </coordinates>
                    </LinearRing>
                </outerBoundaryIs>
            </Polygon>"""^^go:kmlLiteral ;

        geo:asGeoJSON """{
                "type": "Polygon",
                "coordinates": [[
                    [153.3610112, -27.0621757],
                    [153.3658177, -27.1990606],
                    [153.421436, -27.3406573],
                    [153.4269292, -27.3607835],
                    [153.4434087, -27.3315078],
                    [153.4183848, -27.2913403],
                    [153.4189391, -27.2039578],
                    [153.4673476, -27.0267166],
                    [153.3610112, -27.0621757]
                ]]
            }"""^^geo:geoJSONLiteral ;

        geo:asDGGS """<https://w3id.org/dggs/auspix> CELLLIST ((R8346031 R8346034
R8346037
            R83460058 R83460065 R83460068 R83460072 R83460073 R83460074 R83460075
R83460076
            R83460077 R83460078 R83460080 R83460081 R83460082 R83460083 R83460084
R83460085
```

```
            R83460086 R83460087 R83460088 R83460302 R83460305 R83460308 R83460320
R83460321
            R83460323 R83460324 R83460326 R83460327 R83460332 R83460335 R83460338
R83460350
            R83460353 R83460356 R83460362 R83460365 R83460380 R83460610 R83460611
R83460612
            R83460613 R83460614 R83460615 R83460617 R83460618 R83460641 R83460642
R83460644
            R83460645 R83460648 R83460672 R83460686 R83463020 R83463021 R834600487
R834600488
            R834600557 R834600558 R834600564 R834600565 R834600566 R834600567
R834600568
            R834600571 R834600572 R834600573 R834600574 R834600575 R834600576
R834600577
            R834600578 R834600628 R834600705 R834600706 R834600707 R834600708
R834600712
            R834600713 R834600714 R834600715 R834600716 R834600717 R834600718
R834601334
            R834601335 R834601336 R834601337 R834601338 R834601360 R834601361
R834601363
            R834601364 R834601366 R834601367 R834601600 R834601601 R834601603
R834601606
            R834601630 R834601633 R834603220 R834603221 R834603223 R834603224
R834603226
            R834603227 R834603250 R834603251 R834603253 R834603256 R834603280
R834603283
            R834603510 R834603511 R834603512 R834603513 R834603514 R834603515
R834603516
            R834603517 R834603540 R834603541 R834603543 R834603544 R834603546
R834603547
            R834603570 R834603573 R834603576 R834603681 R834603682 R834603684
R834603685
            R834603687 R834603688 R834603810 R834603830 R834603831 R834603832
R834603833
            R834603834 R834603835 R834603836 R834603837 R834603860 R834603861
R834603863
            R834603864 R834603866 R834603867 R834606021 R834606022 R834606024
R834606025
            R834606028 R834606052 R834606055 R834606160 R834606161 R834606162
R834606164
            R834606165 R834606167 R834606168 R834606200 R834606203 R834606206
R834606230
            R834606233 R834606236 R834606260 R834606263 R834606266 R834606401
R834606402
            R834606405 R834606408 R834606432 R834606471 R834606472 R834606474
R834606475
            R834606477 R834606478 R834606500 R834606503 R834606506 R834606530
R834606533
            R834606536 R834606560 R834606563 R834606566 R834606712 R834606715
R834606718
            R834606750 R834606751 R834606752 R834606753 R834606754 R834606755
```

```
R834606757
            R834606758 R834606781 R834606782 R834606784 R834606785 R834606788
R834606800
            R834606803 R834606806 R834606807 R834606830 R834606831 R834606833
R834606834
            R834606835 R834606836 R834606837 R834606838 R834606870 R834606873
R834606874
            R834606876 R834606877 R834630122 R834630125 R834630226 R834630230
R834630231
            R834630232 R834630234 R834630235 R834630237 R834630238 R834630240
R834630241
            R834630242 R834630243 R834630244 R834630245 R834630246 R834630247
R834630261
            R834630262 R834630264 R834630265 R834630268 R834630270 R834630271
R834630273
            R834630276 R834630502))""""^^geo:dggsLiteral ;
    ] ;
.
```

[14] http://www.qudt.org

B.2 Example SPARQL Queries & Rules

New Features checklist - to be removed when all examples are complete:

New element	Section
Non-topological Query Functions	
maxX	Section 8.5.11
maxY	Section 8.5.12
maxZ	Section 8.5.13
minX	Section 8.5.14
minY	Section 8.5.15
minZ	Section 8.5.16
Spatial Aggregate Functions	
BBOX	[Function: geosaf:BBOX]
BoundingCircle	[Function: geoaf:BoundingCircle]
Centroid	[Function: geoaf:Centroid]
ConcatLines	[Function: geoaf:ConcatLines]
ConcaveHull	[Function: geoaf:ConcaveHull]
ConvexHull	[Function: geoaf:ConvexHull]
Union	[Function: geoaf:Union]

This Section provides example data and then illustrates the use of GeoSPARQL functions and the application of rules with that data.

B.2.1 Example Data

The following RDF data (Turtle format) encodes application-specific spatial data. The resulting spatial data is illustrated in Figure 3. The RDF statements define the feature class `my:PlaceOfInterest`, and two properties are created for associating geometries with features: `my:hasExactGeometry` and `my:hasPointGeometry`. `my:hasExactGeometry` is designated as the default geometry for the `my:PlaceOfInterest` feature class.

All the following examples use the parameter values relation_family = Simple Features, serialization = WKT, and version = 1.0.

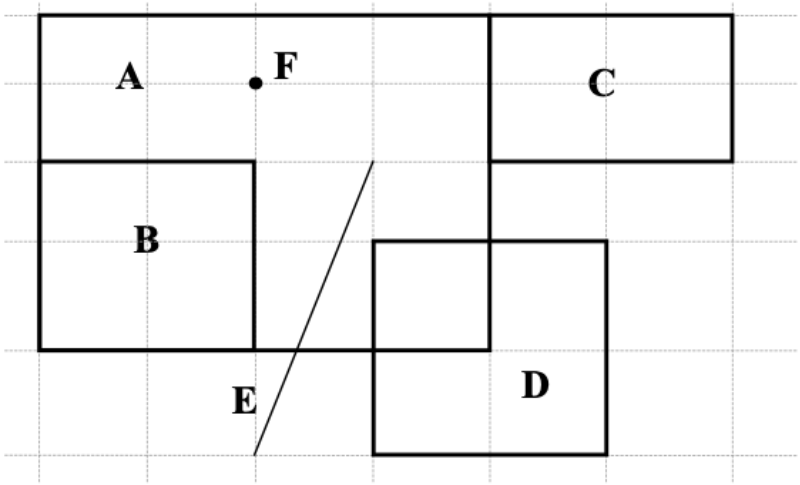

Figure 3. Illustration of spatial data

```
@prefix geo: <http://www.opengis.net/ont/geosparql#> .
@prefix my: <http://example.org/ApplicationSchema#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix sf: <http://www.opengis.net/ont/sf#> .

my:PlaceOfInterest a rdfs:Class ;
    rdfs:subClassOf geo:Feature .

my:A a my:PlaceOfInterest ;
    my:hasExactGeometry my:AExactGeom ;
    my:hasPointGeometry my:APointGeom .

my:B a my:PlaceOfInterest ;
    my:hasExactGeometry my:BExactGeom ;
    my:hasPointGeometry my:BPointGeom .

my:C a my:PlaceOfInterest ;
    my:hasExactGeometry my:CExactGeom ;
    my:hasPointGeometry my:CPointGeom .

my:D a my:PlaceOfInterest ;
    my:hasExactGeometry my:DExactGeom ;
    my:hasPointGeometry my:DPointGeom .

my:E a my:PlaceOfInterest ;
    my:hasExactGeometry my:EExactGeom .

my:F a my:PlaceOfInterest ;
    my:hasExactGeometry my:FExactGeom .

my:hasExactGeometry a rdf:Property ;
    rdfs:subPropertyOf geo:hasDefaultGeometry,
        geo:hasGeometry .

my:hasPointGeometry a rdf:Property ;
```

```
    rdfs:subPropertyOf geo:hasGeometry .

my:AExactGeom a sf:Polygon ;
    geo:asWKT """<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
                 Polygon((-83.6 34.1, -83.2 34.1, -83.2 34.5,
                 -83.6 34.5, -83.6 34.1))"""^^geo:wktLiteral.

my:APointGeom a sf:Point ;
    geo:asWKT """<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
                 Point(-83.4 34.3)"""^^geo:wktLiteral.

my:BExactGeom a sf:Polygon ;
    geo:asWKT """<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
                 Polygon((-83.6 34.1, -83.4 34.1, -83.4 34.3,
                 -83.6 34.3, -83.6 34.1))"""^^geo:wktLiteral.

my:BPointGeom a sf:Point ;
    geo:asWKT """<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
                 Point(-83.5 34.2)"""^^geo:wktLiteral.

my:CExactGeom a sf:Polygon ;
    geo:asWKT """<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
                 Polygon((-83.2 34.3, -83.0 34.3, -83.0 34.5,
                 -83.2 34.5, -83.2 34.3))"""^^geo:wktLiteral.

my:CPointGeom a sf:Point ;
    geo:asWKT """<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
                 Point(-83.1 34.4)"""^^geo:wktLiteral.

my:DExactGeom a sf:Polygon ;
    geo:asWKT """<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
                 Polygon((-83.3 34.0, -83.1 34.0, -83.1 34.2,
                 -83.3 34.2, -83.3 34.0))"""^^geo:wktLiteral.

my:DPointGeom a sf:Point ;
    geo:asWKT """<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
                 Point(-83.2 34.1)"""^^geo:wktLiteral.

my:EExactGeom a sf:LineString ;
    geo:asWKT """<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
                 LineString((-83.4 34.0, -83.3 34.3))"""^^geo:wktLiteral.

my:FExactGeom a sf:Point ;
    geo:asWKT """<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
                 Point(-83.4 34.4)"""^^geo:wktLiteral.
```

B.2.2 Example Queries

This Section illustrates the use of GeoSPARQL functions through a series of example queries.

Example 1: *Find all features that feature* `my:A` *contains, where spatial calculations are based on* `my:hasExactGeometry`.

```
PREFIX my: <http://example.org/ApplicationSchema#>
PREFIX geo: <http://www.opengis.net/ont/geosparql#>
PREFIX geof: <http://www.opengis.net/def/function/geosparql/>

SELECT ?f
WHERE {
    my:A my:hasExactGeometry ?aGeom .
    ?aGeom geo:asWKT ?aWKT .
    ?f my:hasExactGeometry ?fGeom .
    ?fGeom geo:asWKT ?fWKT .

    FILTER (
        geof:sfContains(?aWKT, ?fWKT) &&
            !sameTerm(?aGeom, ?fGeom)
        )
)
```

Result:

?f
my:B
my:F

Example 2: *Find all features that are within a transient bounding box geometry, where spatial calculations are based on* `my:hasPointGeometry`.

```
PREFIX my: <http://example.org/ApplicationSchema#>
PREFIX geo: <http://www.opengis.net/ont/geosparql#>
PREFIX geof: <http://www.opengis.net/def/function/geosparql/>

SELECT ?f
WHERE {
    ?f my:hasPointGeometry ?fGeom .
    ?fGeom geo:asWKT ?fWKT .
    FILTER (
        geof:sfWithin(
            ?fWKT,
            "<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
            Polygon ((-83.4 34.0, -83.1 34.0,
                        -83.1 34.2, -83.4 34.2,
                        -83.4 34.0))"^^geo:wktLiteral
        )
    )
}
```

Result:

?f
my:D

Example 3: *Find all features that touch the union of feature* `my:A` *and feature* `my:D`*, where computations are based on* `my:hasExactGeometry`*.*

```
PREFIX my: <http://example.org/ApplicationSchema#>
PREFIX geo: <http://www.opengis.net/ont/geosparql#>
PREFIX geof: <http://www.opengis.net/def/function/geosparql/>

SELECT ?f
WHERE {
    ?f my:hasExactGeometry ?fGeom .
    ?fGeom geo:asWKT ?fWKT .
    my:A my:hasExactGeometry ?aGeom .
    ?aGeom geo:asWKT ?aWKT .
    ?my:D my:hasExactGeometry ?dGeom .
    ?dGeom geo:asWKT ?dWKT .
    FILTER (
        geof:sfTouches(
            ?fWKT,
            geof:union(?aWKT, ?dWKT)
        )
    )
}
```

Result:

?f
my:C

Example 4: *Find the 3 closest features to feature my:C, where computations are based on* `my:hasExactGeometry`*.*

```
PREFIX uom: <http://www.opengis.net/def/uom/OGC/1.0/>
PREFIX my: <http://example.org/ApplicationSchema#>
PREFIX geo: <http://www.opengis.net/ont/geosparql#>
PREFIX geof: <http://www.opengis.net/def/geosparql/function>

SELECT ?f
WHERE {
    my:C my:hasExactGeometry ?cGeom .
    ?cGeom geo:asWKT ?cWKT .
    ?f my:hasExactGeometry ?fGeom . ?fGeom geo:asWKT ?fWKT .
    FILTER (?fGeom != ?cGeom)
}
ORDER BY ASC (geof:distance(?cWKT, ?fWKT, uom:metre))
LIMIT 3
```

Result:

?f
my:A
my:D
my:E

B.2.3 Example Rule Application

This section illustrates the query transformation strategy for implementing GeoSPARQL rules.

Example 5: *Find all features or geometries that overlap feature* `my:A`.

Original Query:

```
PREFIX geo: <http://www.opengis.net/ont/geosparql#>

SELECT ?f
WHERE { ?f geo:sfOverlaps my:A }
```

Transformed Query (application of transformation rule geor:sfOverlaps):

```
PREFIX my: <http://example.org/ApplicationSchema#>
PREFIX geo: <http://www.opengis.net/ont/geosparql#>
PREFIX geof: <http://www.opengis.net/def/function/geosparql/>

SELECT ?f
WHERE {
    { # check for asserted statement
        ?f geo:sfOverlaps my:A }
    UNION
    { # feature  feature
        ?f geo:hasDefaultGeometry ?fGeom .
        ?fGeom geo:asWKT ?fSerial .
        my:A geo:hasDefaultGeometry ?aGeom .
        ?aGeom geo:asWKT ?aSerial .
        FILTER (geof:sfOverlaps(?fSerial, ?aSerial))
    }
    UNION
    { # feature  geometry
        ?f geo:hasDefaultGeometry ?fGeom .
        ?fGeom geo:asWKT ?fSerial .
        my:A geo:asWKT ?aSerial .
        FILTER (geof:sfOverlaps(?fSerial, ?aSerial))
    }
    UNION
    { # geometry  feature
        ?f geo:asWKT ?fSerial .
        my:A geo:hasDefaultGeometry ?aGeom .
        ?aGeom geo:asWKT ?aSerial .
        FILTER (geof:sfOverlaps(?fSerial, ?aSerial))
    }
    UNION
    { # geometry  geometry
        ?f geo:hasDefaultGeometry ?fGeom .
        ?fGeom geo:asWKT ?fSerial .
        my:A geo:hasDefaultGeometry ?aGeom .
        ?aGeom geo:asWKT ?aSerial .
        FILTER (geof:sfOverlaps(?fSerial, ?aSerial))
    }
}
```

Result:

?f
my:D
my:DExactGeom
my:E
my:EExactGeom

B.2.4 Example Geometry Serialization Conversion Functions

New Features checklist - to be removed when all examples are complete:

New element	Section
Geometry Serializations	
asWKT function	Section 8.4.1.3
asGML function	Section 8.4.2.3
asGeoJSON function	Section 8.4.3.3
asKML function	Section 8.4.4.3

B.1.2.2.1 `geof:asWKT`

For the geometry literal values in B.1.2.3 Geometry Serializations:

Application of the function `geof:asWKT` to the GML, KML, GeoJSON and DGGS literals should return WKT literal and similarly for each of the other conversion methods, `geof:asGML`, `geof:asKML`, `geof:asGeoJSON` & `geof:asDGGS`.

Note that the application of `geof:asDGGS` requires a `dggsIri` parameter which indicates the IRI of the particular DGGS being converted to. In the case of B.1.2.3 Geometry Serializations, this value would be `https://w3id.org/dggs/auspix`, the IRI of the AusPIX DGGS.

Bibliography

- [QUAL] Cohn, Anthony G.; Brandon Bennett; John Gooday; Nicholas Mark Gotts, *Qualitative Spatial Representation and Reasoning with the Region Connection Calculus*, GeoInformatica, 1, 275–316 (1997) https://doi.org/10.1023/A:1009712514511
- [FORMAL] Egenhofer, M. (1989) A Formal Definition of Binary Topological Relationships. In: Litwin, W. and Schek, H.J., Eds., Proceedings of the 3rd International Conference on Foundations of Data Organization and Algorithms (FODO), Paris, France, Lecture Notes in Computer Science, 367, (Springer-Verlag, New York, 1989) 457-472. (1989)
- [CATEG] Egenhofer, Max and J. Herring *Categorizing Binary Topological Relations Between Regions, Lines, and Points*, Geographic Databases, Technical Report, Department of Surveying Engineering, University of Maine (1990)
- [ISO13249] International Organization for Standardization/International Electrotechnical Commission 13249-3, *ISO/IEC 13249-3: Information technology — Database languages — SQL multimedia and application packages — Part 3: Spatial* (2000)
- [ISO19105] International Organization for Standardization, *ISO 19105: Geographic information – Conformance and testing* (2000)
- [ISO19107] International Organization for Standardization, *ISO 19107: Geographic information — Spatial schema* (2003)
- [ISO19109] International Organization for Standardization, *ISO 19109: Geographic information — Rules for application schemas* (2005)
- [ISO19156] International Organization for Standardization, *ISO 19156: Geographic information — Observations and measurements* (2011)
- [LOGIC] Randell, D. A., Cui, Z. and Cohn, A. G.: A spatial logic based on regions and connection, Proc. 3rd Int. Conf. on Knowledge Representation and Reasoning, Morgan Kaufmann, San Mateo, pp. 165–176 (1992)
- [TURTLE] World Wide Web Consortium, *RDF 1.1 Turtle - Terse RDF Triple Language*, W3C Recommendation (25 February 2014). https://www.w3.org/TR/turtle/
- [RDFXML] World Wide Web Consortium, *RDF 1.1 XML Syntax*, W3C Recommendation (25 February 2014). https://www.w3.org/TR/rdf-syntax-grammar/
- [RDF] World Wide Web Consortium, *RDF 1.1 Concepts and Abstract Syntax*, W3C Recommendation (25 February 2014). https://www.w3.org/TR/rdf11-concepts/
- [RDFSEM] World Wide Web Consortium, *RDF 1.1 Semantics*, W3C Recommendation (25 February 2014). https://www.w3.org/TR/rdf11-mt/
- [RDFS] World Wide Web Consortium, *RDF Schema 1.1*, W3C Recommendation (25 February 2014). https://www.w3.org/TR/rdf-schema/
- [RIF] World Wide Web Consortium, *RIF Overview (Second Edition)*, W3C Working Group Note (5 February 2013). https://www.w3.org/TR/rif-overview/
- [OWL2] World Wide Web Consortium, *OWL 2 Web Ontology Language Document Overview (Second Edition)*, W3C Recommendation (11 December 2012). https://www.w3.org/TR/owl2-overview/

- [XML] World Wide Web Consortium, *Extensible Markup Language (XML) 1.0 (Fifth Edition)*, W3C Recommendation (26 November 2008). http://www.w3.org/TR/xml/
- [XMLNS] World Wide Web Consortium, *Namespaces in XML 1.0 (Third Edition)*, W3C Recommendation (8 December 2009). https://www.w3.org/TR/xml-names/
- [XSD1] World Wide Web Consortium, *XML Schema Part 1: Structures (Second Edition)*, W3C Recommendation (28 October 2004). https://www.w3.org/TR/xmlschema-1/
- [XSD2] World Wide Web Consortium, *XML Schema Part 2: Datatypes (Second Edition)*, W3C Recommendation (28 October 2004). https://www.w3.org/TR/xmlschema-2/
- [DGGSAS] Open Geospatial Consortium, *Abstract Standard Topic 21 - Discrete Global Grid Systems - Part 1 Core Reference system and Operations and Equal Area Earth Reference System*, Open Geospatial Consortium Standard (2021) http://www.opengis.net/doc/AS/dggs/2.0
- [IETF5234] Internet Engineering Task Force, *RFC 5234: Internet Standard 68: Augmented BNF for Syntax Specifications: ABNF*. IETF Request for Comment (2008) https://tools.ietf.org/html/std68
- [GEOJSON] Internet Engineering Task Force, *RFC 7946: The GeoJSON Format*. IETF Request for Comment (August 2016). https://tools.ietf.org/html/rfc7946
- [OGCKML] Open Geospatial Consortium, *OGC KML 2.3*. OGC Implementation Standard (04 August 2015). http://www.opengis.net/doc/IS/kml/2.3
- [ISO19125-1] International Organization for Standardization, *ISO 19125-1: Geographic information — Simple feature access — Part 1: Common architecture*
- [OGC07-036] Open Geospatial Consortium, *OGC 07-036: Geography Markup Language (GML) Encoding Standard*, Version 3.2.1 (27 August 2007).
- [IETF3987] Internet Engineering Task Force, *RFC 3987: Internationalized Resource Identifiers (IRIs)*. IETF Request for Comment (January 2005). https://tools.ietf.org/html/rfc3987
- [RIFCORE] World Wide Web Consortium, *RIF Core Dialect (Second Edition)*, W3C Recommendation (5 February 2013) http://www.w3.org/TR/rif-core/
- [SPARQL] World Wide Web Consortium, *SPARQL 1.1 Query Language*, W3C Recommendation (21 March 2013). https://www.w3.org/TR/sparql11-query/
- [SPARQLENT] World Wide Web Consortium, *SPARQL 1.1 Entailment Regimes*, W3C Recommendation (21 March 2013). https://www.w3.org/TR/sparql11-entailment/
- [SPARQLPROT] World Wide Web Consortium, *SPARQL 1.1 Protocol*, W3C Recommendation (21 March 2013)> http://www.w3.org/TR/sparql11-protocol/
- [SPARQLRESX] World Wide Web Consortium, *SPARQL Query Results XML Format (Second Edition)*, W3C Recommendation (21 March 2013). https://www.w3.org/TR/rdf-sparql-XMLres/
- [SPARQLRESJ] World Wide Web Consortium, *SPARQL 1.1 Query Results JSON Format*, W3C Recommendation (21 March 2013). http://www.w3.org/TR/sparql11-results-json/
- [SHACL] World Wide Web Consortium, *Shapes Constraint Language (SHACL)*, W3C Recommendation (20 July 2017). https://www.w3.org/TR/shacl/
- [PROF] World Wide Web Consortium, *The Profiles Vocabulary*, W3C Working Group Note (18 December 2019). https://www.w3.org/TR/dx-prof/
- [SKOS] World Wide Web Consortium, *SKOS Simple Knowledge Organization System Reference*,

W3C Recommendation (18 August 2009). https://www.w3.org/TR/skos-reference/

- [JSON-LD] World Wide Web Consortium, *JSON-LD 1.1: A JSON-based Serialization for Linked Data*, W3C Recommendation (16 July 2020). https://www.w3.org/TR/json-ld11/
- [CHARTER] Open Geospatial Consortium, *OGC GeoSPARQL SWG Charter*, OGC Working Group Charter (25 August 2020). https://github.com/opengeospatial/ogc-geosparql/blob/master/charter/swg_charter.pdf
- Clementini, Eliseo; Di Felice, Paolino and van Oosterom, Peter, *A small set of formal topological relationships suitable for end-user interaction* (1993). In Abel, David; Ooi, Beng Chin (eds.). Advances in Spatial Databases: Third International Symposium, SSD '93 Singapore, June 23–25, 1993 Proceedings. Lecture Notes in Computer Science. 692/1993. Springer. pp. 277–295. doi:10.1007/3-540-56869-7_16.